

SHEIN 服装 PLM

+ Multi-Agent AI 智能体

完整业务解决方案 -- 从趋势采集到热销复盘，全链路自动化
Multi-Agent MCP 工具协议 三层 Memory LangGraph PLM 深度耦合

方案设计：马永 | 2026 年 6 月

执行摘要

核心业务目标

用 Multi-Agent 智能体 打通 PLM 全流程，完成 趋势采集 to 选品定款 to 款式生成 to 成本校验 to 热销复盘 to 数据回流迭代 全自动化。

量化指标

指标	现状	目标
选品耗时	3-5天/款	2小时内
人工重复工作	趋势采集/成本核算/录入	替代70%
爆款率	依赖个人经验	AI辅助+30%
PLM数据一致性	人工录入易出错	零差错自动同步

LangGraph 多智能体编排 + MCP 标准化工具协议 + 三层 Memory + PLM 深度耦合

业务完整故事：真实选品全链路

角色

选品设计师 | 供应链专员 | 运营复盘专员 | AI 产品经理

9 步闭环流程

Step 1

设计师上传社媒穿搭图，输入：“生成3款2026夏季辣妹牛仔短裤，匹配近3个月热销版型，成本≤35元”

Step 2

总管Agent解析需求，拆解为4个子任务，分发给4个专业Agent并行执行

Step 3

趋势Agent：MCP爬取Instagram/Pinterest/TikTok潮流+向量库RAG检索流行元素

Step 4

PLM数据Agent：MCP读取历史款销量/退货/面料BOM/工厂成本

Step 5

多模态绘图Agent：生成效果图+拆解版型/面料/辅料

Step 6

供应链校验Agent：核算成本，拦截超预算款式，返回修改方案

Step 7

汇总Agent：整合输出选品报告+款式图，一键写入PLM生成工艺单

Step 8

复盘Agent：上新2周后拉取销售数据，总结热销/滞销特征，生成训练集

Step 9

模型进化：训练数据入库，增量微调行业大模型

行业痛点与为什么是AI Agent

6大痛点

痛点	损失
趋势采集依赖人工刷社媒 1-2天	错过潮流窗口期
销售/成本/BOM数据分散多个系统	选品缺乏全局数据
从概念到效果图 3-5天	SKU受限
款式确定后才核算成本	返工浪费排期
复盘靠Excel，经验无法结构化	每次从头来
设计师离职带走爆款经验	组织记忆缺失

为什么不是普通工作流？

RAG Chatbot 不够：只能回答问题，不能"做事情"（查库/调模型/写PLM）

拖拽工作流不够：线性流程无法处理多约束并行决策

Multi-Agent + MCP 是答案：每个Agent专注一个领域，MCP标准化工具调用，总管Agent动态编排--这正是Anthropic/OpenAI 2026主打范式

四层技术架构总览

第1层：前端业务应用层

Go 仪表盘
JD要求的dashboard
React 前端
PLM内嵌面板
Celery 任务队列
异步批量任务
RBAC 权限
角色数据隔离

第2层：Multi-Agent 编排层（核心）

LangGraph
Agent图编排
AutoGen
多Agent对话
5类Agent
总管/趋势/数据/绘图/校验
三层Memory
工作/会话/长期

第3层：MCP 标准化工具层

数据库MCP
PLM/销售/BOM
多模态MCP
文生图/CLIP
爬虫MCP
社媒数据抓取
PLM同步MCP
自动写回款式

第4层：底层数据&模型算力层

MySQL/PostgreSQL
PLM业务主数据
Milvus/Chroma
多模态向量检索
DeepSeek/Claude
推理+规划
数据治理管线
训练集自动生成

全业务流程泳道图（分角色+步骤+对应技术）

泳道1：设计师（业务发起端）

步骤	操作	技术实现
1	在PLM内嵌AI面板输入选品需求 + 上传参考穿搭图	Go Web仪表盘 + 多模态图片预处理（resize/format/EXIF清洗）
2	等待Agent输出选品方案，人工审核调整参数（成本、风格）	会话记忆缓存本次需求上下文，无需重复输入
3	确认方案，一键写入PLM系统	MCP PLM同步服务 to 创建款式档案 + 生成BOM工艺单

泳道2：Multi-Agent编排中心（核心流程）

步骤	操作	技术实现
1	规划Agent拆解任务，分发4个子Agent并行执行	LangGraph多智能体调度 + Plan任务拆解逻辑 + JSON Schema结构化输出
2	各Agent通过MCP工具层调用外部资源，并行返回结果	MCP标准化工具调用 + Redis会话共享上下文
3	汇总Agent校验所有数据，执行业务硬规则	业务规则引擎 + LLM结果校验 + 自动纠错重跑
4	生成完整选品报告+款式效果图，写入长期记忆库	Mem0长期记忆 + 多模态PDF生成pipeline

泳道3：MCP工具执行层（数据读写/模型调用）

步骤	操作	技术实现
1	MCP数据库服务读取PLM历史款销量、物料成本	MCP数据库网关 + SQL脱敏审计 + 多库异构数据对齐
2	MCP多模态服务生成新款效果图、拆解款式属性	多模态模型推理流水线 + CLIP图像特征提取
3	MCP同步服务将最终方案回写PLM系统	PLM开放API + 数据字段映射中间件

泳道4：底层数据&模型迭代（后置复盘）

步骤	操作	技术实现
1	上新2周后，复盘Agent定时拉取销售数据	Celery定时任务 + 数据湖批量读取
2	自动清洗热销/滞销案例，生成训练数据集入库	数据治理脚本 + 向量库新增行业样本
3	增量微调行业大模型，优化选品泛化能力	RAG知识库更新 + 模型微调管线（LoRA/QLoRA）

JD 技术要求匹配对照表

典型 JD 要求 × 方案能力映射

基于 SHEIN/服装行业 AI 产品经理 & 技术负责人 JD，逐条映射方案能力。

#	JD 常见要求	我们的方案能力	对应技术实现
1	「熟悉服装PLM系统，了解选品/打版/生产全流程」	PLM-MCP 深度耦合：4 类 MCP 工具直接读写 PLM 数据库（SKU/BOM/物料/工厂），支持款式创建、规格同步、成本校验全自动化	PLM-MCP Server (Python) + PostgreSQL + PLM REST API 适配层
2	「有 AI Agent/Multi-Agent 系统设计经验」	7 个专业 Agent 协作：总管Agent、趋势Agent、PLM数据Agent、多模态绘图Agent、成本校验Agent、运营复盘Agent、人机协作Agent	LangGraph StateGraph + Supervisor Pattern + 条件路由
3	「熟悉 LLM 应用开发，LangChain/LlamaIndex」	全栈 LLM 应用：LangChain 消息管理 + LangGraph 编排 + LlamaIndex RAG 向量检索	LangChain ChatModels + LangGraph + LlamaIndex VectorStoreIndex
4	「了解 RAG/向量数据库/知识图谱」	三层 Memory 中的 L3 长期记忆：Qdrant 向量检索 + Zep 图记忆引擎 + NetworkX 知识图谱	Qdrant + Zep + PostgreSQL + pgvector + NetworkX
5	「有 MLOps/模型部署经验」	Docker Compose + K8s 部署，多模态模型 (Flux/CLIP) 通过 ComfyUI MCP 暴露，模型版本管理 + A/B 测试	Docker Compose / K8s Deployment + ComfyUI MCP + MLflow
6	「熟悉多模态 AI：文生图/图生图/图像理解」	Media-MCP 完整多模态 Pipeline：Flux 文生图 + CLIP 风格提取 + SAM 服装分割 + ControlNet 版型控制	Flux.1-dev + CLIP + SAM + ControlNet + ComfyUI Workflow
7	「有微服务/分布式系统设计经验」	MCP 协议天然微服务化：4 个独立 MCP Server 可独立扩缩容、独立部署、独立版本管理	MCP stdio to 升级为 SSE/HTTP Transport + K8s Service
8	「了解时尚趋势分析方法论」	Trend-MCP 自动采集 Instagram/Pinterest/TikTok/Vogue Runway 趋势，情感分析 + 趋势预测	Trend-MCP + 社媒 API + LLM 趋势总结 + 向量 RAG
9	「有数据驱动决策/Analytics 经验」	Ops-MCP 销售数据分析 + 智能复盘：自动对比 预测 vs 实际销售，提取关键设计洞察回写 L3	Ops-MCP + ClickHouse + Prophet 时序预测 + LLM 洞察提取
10	「具备产品从0到1的落地能力」	完整 4 阶段实施路线图 (Phase 1-4)，MVP 8周可上线，含 Docker 一键部署脚本	Phase 1: 基础设施 + Phase 2: 核心流程 + Phase 3: 智能增强 + Phase 4: 全面推

11	「熟悉 Python/后端开发」	全 Python 技术栈：LangGraph + MCP Server + FastAPI + SQLAlchemy + httpx	Python 3.11+ / asyncio / FastAPI / Pydantic v2
12	「有敏捷开发/跨团队协作经验」	MCP 协议天然解耦团队边界：趋势团队维护 Trend-MCP、PLM 团队维护 PLM-MCP、AI 团队维护编排层	MCP 标准化接口 + Git 多仓库 + CI/CD 独立流水线
13	「了解服装供应链/生产流程」	PLM-MCP 深度对接供应商/工厂数据：产能查询、面料库存、交期预估	PLM-MCP.get_supplier_capacity + ERP 接口对接
14	「熟悉监控/告警/可观测性」	LangSmith 链路追踪 + Prometheus + Grafana 监控 + Sentry 错误追踪 + MCP Server 健康检查	LangSmith SDK + OpenTelemetry + Prometheus + Grafana
15	「有 AI Safety/内容审核经验」	多模态内容审核 Agent：生成图片自动过 CLIP 安全检测 + 人工二次审核 + 审计日志	CLIP Safety Classifier + 人工审核队列 + 审计 MCP

技术栈能力矩阵

能力域	核心技术	方案覆盖度	熟练度要求
Agent 编排	LangGraph, LangChain	100%	高级
MCP 协议	mcp (Python SDK), JSON-RPC 2.0	100%	高级
Memory 系统	Mem0, Zep, Redis, Qdrant	100%	高级
多模态 AI	Flux, CLIP, SAM, ComfyUI	95%	中高级
RAG/向量检索	LlamaIndex, Qdrant, pgvector	100%	中级
PLM 系统对接	PLM REST API, PostgreSQL	80%	中级
部署/Ops	Docker, K8s, Prometheus	100%	中高级
数据管线	Airflow, dbt, ClickHouse	90%	中级

匹配结论：本方案完整覆盖 SHEIN 服装行业 AI 产品经理/技术负责人 JD 中 100% 的核心技术要求，并提供了可直接落地的技术实现方案和代码示例。

为什么不是拖拽 workflow？LLM-Native 编排 vs 传统 workflow

核心命题

市面上有很多「AI 工作流」产品（Dify、Coze、扣子等）提供了拖拽式界面来编排 LLM 调用。为什么我们的方案选择了 LangGraph 代码级编排而非拖拽 workflow？这是基于服装 PLM 场景的深度思考。

决定性对比

对比维度	拖拽 workflow (Dify/Coze)	LangGraph 代码编排 (本方案)	PLM 场景的胜负手
动态路由	有限的 if/else 分支，节点连接在编辑时固定	LLM 根据运行时 State 动态决定下一步： add_conditional_edges()	设计师说「成本超了」to 自动路由到面料替换节点；说「风格不对」to 路由到重绘节点。拖拽 workflow 无法预定义所有分支组合
人机协作循环	需要人工审批节点打断流程，但难以实现「改完继续自动流转」	interrupt() 挂起等人工输入 + 恢复后自动判断下一步	设计师改完腰头尺寸后，Agent 自动判断是否需要重新校验成本/重新生成效果图
多 Agent 并行+汇聚	并行节点有限，汇聚逻辑简单（等全部完成）	send() API 动态 Fork 多个 Agent、operator.add 汇聚结果	一个需求同时发 Trend/PLM/Media/Ops 4 个 Agent，部分完成即可开始下一步，不等最慢的那个
State 管理深度	扁平的变量传递，无类型系统	TypedDict + Pydantic 强类型 State，支持嵌套结构、Annotated reducer	PLM 场景 State 包含 20+ 字段（设计稿、成本、BOM、趋势、审核意见），拖拽 workflow 的变量系统无法驾驭
可测试性	手动点击测试，无法 CI 集成	单元测试 + 集成测试 + LangSmith 评估框架	选品 Agent 的准确率需要持续回归测试，拖拽 workflow 无法 CI/CD
版本管理	平台内版本，难以 diff/review	Git 版本管理，PR review，分支开发	企业级协作必需：多个工程师并行开发不同 Agent，通过 PR 合入主干
工具生态	平台内置工具，扩展需等平台支持	MCP 协议标准化，任何语言实现的 MCP Server 均可接入	PLM 的 REST API、ERP 的 SOAP 接口——这些均需定制 MCP Server，拖拽 workflow 平台不支持
复杂度上限	节点数 > 50 时画布混乱，难以维护	代码模块化：每个 Agent 一个独立文件，函数级别复用	本方案目前 7 个 Agent、20+ 节点、4 个 MCP Server——已经远超拖拽 workflow 的舒适区
定制化程度	受平台能力限制，黑盒 LLM 调用	完全可控：自定义 prompt 模板、自定义 tool 执行逻辑、自定义 retry 策略	服装行业需要大量领域特定逻辑（面料替换规则、成本核算公式）——这些无法在拖拽平台表达
延迟优化	平台固定调用模式，无法精细控制	asyncio 协程级并行、流式输出、智能缓存	4 个 Agent 并行调用 MCP，节省 70% 总延迟——拖拽 workflow 的「并行节点」难以实现真正的协程并行

拖拽工作流的合理边界

我们并不完全否定拖拽工作流--它在以下场景仍然有价值：

场景	适合拖拽工作流	不适合拖拽工作流
简单 RAG 问答	✔ 知识库 + LLM 回答	
固定流程自动化	✔ 日报生成、定时摘要	
非技术人员使用	✔ 运营人员自助搭建	
PLM 智能选品		✘ 动态路由 + 多Agent + 人机协作
多模态设计生成		✘ 复杂 State + 图像 Pipeline
成本自动校验		✘ 需调用 PLM 定制接口

LangGraph 编排核心代码

```
# graph_builder.py - LangGraph 优雅表达复杂编排逻辑
from langgraph.graph import StateGraph, END
from langgraph.checkpoint.memory import MemorySaver
from langgraph.types import interrupt, Send

builder = StateGraph(PLMAgentState)

# 注册节点
builder.add_node("supervisor", supervisor_agent)      # 总管：分析需求、分派任务
builder.add_node("trend_agent", trend_agent)         # 趋势分析
builder.add_node("plm_agent", plm_agent)             # PLM 数据查询
builder.add_node("media_agent", media_agent)         # 多模态生成
builder.add_node("ops_agent", ops_agent)             # 运营数据
builder.add_node("cost_checker", cost_checker_agent) # 成本校验
builder.add_node("human_review", human_review_node)  # 人机协作
builder.add_node("plm_writer", plm_write_agent)      # 写入 PLM

builder.set_entry_point("supervisor")

# 🔥 核心：LLM 动态路由--拖拽工作流做不到的
builder.add_conditional_edges(
    "supervisor",
    lambda state: state["next_agents"], # LLM 根据需求决定启动哪些 Agent
    {
        "trend": "trend_agent",
        "plm": "plm_agent",
        "media": "media_agent",
        "ops": "ops_agent",
        "finish": END
    }
)

# 每个 Agent 完成后回到 supervisor 判断
builder.add_edge("trend_agent", "supervisor")
builder.add_edge("plm_agent", "supervisor")
builder.add_edge("media_agent", "supervisor")
builder.add_edge("ops_agent", "supervisor")

# 🔥 人机协作：动态中断等待审核
builder.add_conditional_edges(
    "cost_checker",
    lambda state: "human_review" if state["needs_human_review"] else "plm_writer",
    {"human_review": "human_review", "plm_writer": "plm_writer"}
)

# 审核通过后写入 PLM，审核不通过回到 supervisor 重新调整
builder.add_conditional_edges(
    "human_review",
    lambda state: "plm_writer" if state["review_approved"] else "supervisor",
    {"plm_writer": "plm_writer", "supervisor": "supervisor"}
)

builder.add_edge("plm_writer", END)

# 编译，带 Memory 检查点
memory = MemorySaver()
graph = builder.compile(checkpointer=memory)

# 执行
result = await graph.ainvoke(
    {"messages": [HumanMessage(content=design_brief)], "task_id": task_id},
    config={"configurable": {"thread_id": session_id}}
)
```

决策总结

标准	拖拽工作流	LangGraph
上手速度	★★★★★★	★★★★★★
复杂流程表达力	★★★	★★★★★★

PLM 场景适配	☆☆☆	☆☆☆☆☆☆
企业级工程化	☆☆	☆☆☆☆☆☆
团队协作	☆☆☆	☆☆☆☆☆☆
长期维护性	☆☆☆	☆☆☆☆☆☆

结论：拖拽 workflow 是「LLM 调用的可视化编排器」，适合简单线性流程。SHEIN PLM 选品场景需要的是 LLM-Native 的智能编排引擎——能根据服装设计的不确定性动态决策、支持多 Agent 并行、处理 20+ 字段的复杂 State、无缝集成企业 PLM 系统。这些只有 LangGraph 能做到。

部署架构与基础设施

Docker Compose 开发环境

```
# docker-compose.yml
version: '3.8'
services:
  # === Agent 编排服务 ===
  agent-orchestrator:
    build: ./agent-orchestrator
    ports: ["8000:8000"]
    environment:
      - LLM_API_KEY=${DEEPSEEK_API_KEY}
      - LLM_BASE_URL=https://api.deepseek.com/v1
      - REDIS_URL=redis://redis:6379
      - MILVUS_HOST=milvus
    depends_on: [redis, milvus]

  # === MCP 数据库网关 ===
  mcp-db-gateway:
    build: ./mcp-servers/db-gateway
    ports: ["8001:8001"]
    environment:
      - PLM_DB_URL=postgres://plm_user:pass@plm-db:5432/plm
      - SALES_DB_URL=mysql://sales_user:pass@sales-db:3306/sales
      - AUDIT_ENABLED=true

  # === MCP 多模态服务 ===
  mcp-multimodal:
    build: ./mcp-servers/multimodal
    ports: ["8002:8002"]
    environment:
      - SD_API_URL=http://stable-diffusion:7860
      - CLIP_MODEL=ViT-L/14
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: 1
              capabilities: [gpu]

  # === MCP 爬虫服务 ===
  mcp-crawler:
    build: ./mcp-servers/crawler
    ports: ["8003:8003"]
    environment:
      - PLAYWRIGHT_BROWSERS_PATH=/ms-playwright
      - PROXY_POOL_URL=http://proxy-pool:5010

  # === Redis 会话缓存 ===
  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]

  # === Milvus 向量数据库 ===
  milvus:
    image: milvusdb/milvus:v2.3.0
    ports: ["19530:19530", "9091:9091"]
    environment:
      - ETCD_ENDPOINTS=etcd:2379
      - MINIO_ADDRESS=minio:9000

  # === PLM 主数据库 ===
  plm-db:
    image: postgres:16
    ports: ["5432:5432"] # 内部，不对外暴露
    environment:
      - POSTGRES_DB=plm
      - POSTGRES_USER=plm_user
      - POSTGRES_PASSWORD=${PLM_DB_PASSWORD}
    volumes:
      - plm_data:/var/lib/postgresql/data

volumes:
  plm_data:
```

Kubernetes 生产环境架构

组件	K8s资源	副本数	资源
----	-------	-----	----

Agent Orchestrator Deployment + HPA	3-10	2CPU 4Gi
MCP DB Gateway Deployment	2-5	1CPU 2Gi
MCP Multimodal Deployment + GPU NodeSelector	2-4	4CPU 16Gi + A10 GPU
MCP Crawler Deployment + CronJob	2	2CPU 4Gi
Redis StatefulSet + Sentinel	3	2CPU 4Gi
Milvus StatefulSet	3	8CPU 32Gi

监控与可观测性

层面	工具	指标
基础设施	Prometheus + Grafana	CPU/内存/GPU利用率、网络IO
应用层	OpenTelemetry + Jaeger	Agent调用链路追踪、MCP工具耗时
业务层	自研Go Dashboard	选品任务数、爆款率、成本节省
告警	AlertManager + 飞书/钉钉	Agent失败率>5%、GPU OOM、DB连接池耗尽

实施路线图：Phase 1 to Phase 4

总体时间规划

Phase	时间	主题	目标	团队
Phase 1	第 1-4 周	🏗️ 基础设施搭建	MCP Server 上线 + PLM 打通	2 BE + 1 AI + 1 DevOps
Phase 2	第 5-8 周	⚡ 核心流程贯通	端到端选品自动化 (MVP)	2 BE + 2 AI + 1 设计师
Phase 3	第 9-12 周	🧠 智能增强	记忆系统 + 微调 + 人机协作优化	2 BE + 2 AI + 1 DevOps
Phase 4	第 13-24 周	🌈 全面推广	全品类覆盖 + 复盘闭环 + 持续优化	全员

Phase 1: 基础设施搭建 (Week 1-4)

Week 1-2: 环境与框架

任务	负责人	交付物	验收标准
Docker Compose 环境搭建	DevOps	一键启动脚本	<code>docker compose up</code> 所有服务正常启动
LangGraph 项目骨架	AI Engineer	项目仓库 + State 定义	空 StateGraph 可编译运行
PLM 数据库 Schema 设计	Backend	DDL + 迁移脚本	Styles/BOM/AuditLog 表创建
MCP SDK 集成	AI Engineer	MCP Client Manager	成功连接一个 MCP Server
CI/CD Pipeline	DevOps	GitHub Actions 配置	PR 自动测试 + 构建镜像

Week 3-4: MCP Server 开发

任务	负责人	交付物	验收标准
Trend-MCP Server	Backend	4 个工具 + 2 类资源	MCP Inspector 可调用全部工具
PLM-MCP Server (读)	Backend	<code>query_sku / get_bom / search_similar</code>	返回真实 PLM 数据
Media-MCP Server (基础)	AI Engineer	<code>generate_design_image</code> (Flux)	输入 prompt 输出效果图
Ops-MCP Server (基础)	Backend	<code>get_sales_performance</code>	返回 BI 销售数据
总管 Agent (Supervisor)	AI Engineer	需求解析 + 任务分发	正确解析需求并路由到子 Agent

Phase 1 里程碑：4 个 MCP Server 全部上线，LangGraph 可调用 MCP 工具。PLM 数据库读写通。

Phase 2: 核心流程贯通 (Week 5-8)

Week 5-6: Agent 开发

任务	负责人	交付物	验收标准
Trend Agent	AI Engineer	趋势分析节点	输入品类返回趋势报告
PLM Data Agent	Backend	PLM 数据查询节点	返回历史 SKU + BOM + 相似款
Media Agent	AI Engineer	多模态生成节点	CLIP 提取 + Flux 生成
Cost Checker Agent	Backend	成本校验节点	自动计算总成本并与预算对比
流程串联	AI Engineer	完整 LangGraph	端到端：需求 to 方案 (手动触发)

Week 7-8: MVP 测试 & 优化

任务	负责人	交付物	验收标准
----	-----	-----	------

PLM-MCP 写回能力 Backend	write_style_spec + 审计日志 写入 PLM 后可查询 + 审计可追溯
人机协作节点	AI Engineer interrupt + 审核流程 设计稿可挂起等人工审核
内测 (5 名设计师)	PM + Designer 使用反馈报告 采纳率 ≥ 60%, NPS ≥ 30
性能优化	All P95 < 5min 端到端选品在 5 分钟内完成
MVP 发布	PM 发布说明 + 使用文档 5 名设计师正式使用

Phase 2 里程碑：MVP 上线！设计师输入需求 to AI 自动生成 3 款方案 to 人工审核 to 一键写入 PLM。5 名内测用户。

Phase 3: 智能增强 (Week 9–12)

Week 9–10: Memory 系统

任务	负责人	交付物	验收标准
L1 工作记忆 (Redis Checkpointer)	AI Engineer	LangGraph State 持久化	进程重启后可恢复任务
L2 会话记忆 (Mem0)	AI Engineer	跨轮对话上下文	设计师改需求后 Agent 记住历史
L3 长期记忆 (Zep + Qdrant)	AI Engineer + BE	知识归档 + 相似设计召回	新会话能召回历史成功案例
知识图谱构建	AI Engineer	面料-款式-趋势图谱	NX 图可视化 + 遍历查询

Week 11–12: 微调 & 优化

任务	负责人	交付物	验收标准
Flux LoRA 微调	AI Engineer	SHEIN 风格 LoRA 生成图	风格趋近 SHEIN 品牌
LLM 指令微调	AI Engineer	服装领域 LoRA	prompt 优化更精准
反馈收集系统	Backend	审核反馈埋点	每个审核操作自动记录
A/B 测试框架	AI Engineer	模型版本路由	10% 流量走新模型
监控 Dashboard	DevOps	Grafana 面板	关键指标实时可见

Phase 3 里程碑：三层 Memory 上线，Agent 具备「记忆」能力。微调模型 A/B 测试中。内测扩展至 20 名设计师。

Phase 4: 全面推广 (Week 13–24)

Week 13–16: 品类扩展

任务	负责人	交付物	验收标准
女装全品类适配	AI + Designer	连衣裙/上衣/外套/裙装	每品类通过 20 次测试
男装品类适配	AI + Designer	T恤/衬衫/裤装/外套	每品类通过 20 次测试
配饰品类适配	AI + Designer	包包/帽子/腰带	基础款式可用

Week 17–20: 闭环建设

任务	负责人	交付物	验收标准
自动复盘 Agent	AI Engineer	定时复盘 + 洞察提取	上线30天自动触发复盘
洞察沉淀 Pipeline	AI Engineer	洞察自动写入 L3	每日自动沉淀 ≥ 5 条洞察
数据治理自动化	Backend	GE 日常检查 + 告警	异常自动告警
RLHF 反馈闭环	AI Engineer	偏好数据自动收集	每周积累 ≥ 100 条偏好对

Week 21–24: 全量推广

任务	负责人	交付物	验收标准
全团队上线	PM	培训 + 文档 + 支持	50+ 设计师日常使用
KPI 归因分析	DS	ROI 验证报告	爆款率 ≥20%, 成本 ≥30%
安全审计	DevOps	安全评估报告	通过安全审查
持续优化机制	All	双周迭代	模型/流程持续改进

Phase 4 里程碑：全品类覆盖，全团队使用。复盘闭环运转，知识飞轮启动。ROI 验证通过，方案全面成功。

风险与应对

风险	概率	影响	应对措施
LLM 生成质量不稳定	🟡 中	高	多次采样 + 人工审核流程 + 微调优化
PLM 接口变更	🟡 中	中	MCP 解耦层 + 版本化接口 + 契约测试
GPU 资源不足	🟢 低	中	云 GPU 弹性扩缩 + 任务队列削峰

- 设计师抵触

● 中高

渐进式引入 + 人机协作模式 + 效率收益展示
- 数据合规风险

● 低高

数据脱敏 + 审计日志 + 法务审查
- 社媒 API 限流/封禁

● 中中

多平台冗余 + 本地缓存 + 手动补充

团队配置建议

角色	Phase 1-2	Phase 3	Phase 4	职责
AI 工程师	1人	2人	2人	LangGraph 编排、模型微调、Prompt 工程
后端工程师	2人	2人	2人	MCP Server、PLM 对接、数据库
DevOps	1人	1人	1人	部署、监控、CI/CD、GPU 管理
AI PM	0.5人	1人	1人	需求管理、指标跟踪、跨团队协作
选品设计师 (种子用户)	2人	5人	全员	使用反馈、审核、训练数据标注
数据工程师 (兼职)	0.5人	0.5人	1人	数据管线、数据质量、BI 对接

教程：用 Python mcp 库实现 4 个 MCP Server

环境准备

```
# 安装 MCP SDK
pip install mcp httpx sqlalchemy asyncpg qdrant-client
```

```
# 项目结构
mcp_servers/
├── trend/
│   ├── trend_mcp_server.py
│   └── requirements.txt
├── plm/
│   ├── plm_mcp_server.py
│   └── requirements.txt
├── media/
│   ├── media_mcp_server.py
│   └── requirements.txt
└── ops/
    ├── ops_mcp_server.py
    └── requirements.txt
```

Server 1: Trend-MCP – 趋势采集服务

```
#!/usr/bin/env python3
"""
Trend-MCP Server: 社交媒体趋势采集与分析
暴露 4 个工具 + 3 类资源
"""
import asyncio
import os
from datetime import datetime, timedelta
from mcp.server import Server
from mcp.server.stdio import stdio_server
from mcp.types import Tool, TextContent, Resource, ResourceTemplate
import httpx

# ===== 初始化 MCP Server =====
server = Server("shein-trend-mcp")

# 配置
INSTAGRAM_TOKEN = os.getenv("INSTAGRAM_TOKEN")
PINTEREST_TOKEN = os.getenv("PINTEREST_TOKEN")
TIKTOK_TOKEN = os.getenv("TIKTOK_TOKEN")

# ===== 工具 1: search_social_trends =====
@server.list_tools()
async def list_tools() -> list[Tool]:
    return [
        Tool(
            name="search_social_trends",
            description="跨平台搜索服装趋势: Instagram/Pinterest/TikTok. 返回帖子/图片/互动数据。",
            inputSchema={
                "type": "object",
                "properties": {
                    "keyword": {
                        "type": "string",
                        "description": "搜索关键词, 如 'Y2K denim shorts summer 2026'"
                    },
                    "platform": {
                        "type": "string",
                        "enum": ["instagram", "pinterest", "tiktok", "all"],
                        "default": "all"
                    },
                    "limit": {
                        "type": "integer",
                        "default": 20,
                        "minimum": 1,
                        "maximum": 100
                    }
                }
            }
        )
    ]
```

```

        },
        "date_range_days": {
            "type": "integer",
            "default": 30,
            "description": "搜索最近 N 天的内容"
        }
    },
    "required": ["keyword"]
},
),
Tool(
    name="fetch_trending_hashtags",
    description="获取指定服装品类的实时热门标签和话题",
    inputSchema={
        "type": "object",
        "properties": {
            "category": {
                "type": "string",
                "enum": ["womenswear", "menswear", "denim", "activewear", "outerwear", "dresses", "accessories"]
            },
            "region": {
                "type": "string",
                "default": "global",
                "enum": ["global", "us", "eu", "asia", "latam"]
            },
            "limit": {"type": "integer", "default": 15}
        },
        "required": ["category"]
    }
),
Tool(
    name="analyze_sentiment",
    description="分析品牌/款式关键词在社交媒体上的情感倾向和声量趋势",
    inputSchema={
        "type": "object",
        "properties": {
            "brand_keyword": {"type": "string", "description": "品牌或款式关键词"},
            "date_range_days": {"type": "integer", "default": 30},
            "compare_with": {
                "type": "array",
                "items": {"type": "string"},
                "description": "可选: 对比关键词列表"
            }
        },
        "required": ["brand_keyword"]
    }
),
Tool(
    name="get_runway_trends",
    description="获取时装周/秀场趋势数据, 按季节和品类筛选",
    inputSchema={
        "type": "object",
        "properties": {
            "season": {
                "type": "string",
                "enum": ["SS25", "AW25", "SS26", "AW26", "resort26"]
            },
            "category": {"type": "string", "default": "all"},
            "design_elements": {
                "type": "array",
                "items": {"type": "string"},
                "description": "筛选特定设计元素: color, silhouette, fabric, print, detail"
            }
        },
        "required": ["season"]
    }
)
)

# ===== 工具实现 =====
@server.call_tool()
async def call_tool(name: str, arguments: dict) -> list[TextContent]:
    if name == "search_social_trends":
        return await _handle_search_social_trends(arguments)
    elif name == "fetch_trending_hashtags":
        return await _handle_fetch_trending_hashtags(arguments)
    elif name == "analyze_sentiment":
        return await _handle_analyze_sentiment(arguments)
    elif name == "get_runway_trends":
        return await _handle_get_runway_trends(arguments)
    else:
        return [TextContent(type="text", text=f"Unknown tool: {name}")]

async def _handle_search_social_trends(args: dict) -> list[TextContent]:
    keyword = args["keyword"]
    platform = args.get("platform", "all")
    limit = args.get("limit", 20)
    date_range = args.get("date_range_days", 30)

    results = []
    async with httpx.AsyncClient(timeout=30.0) as client:
        # 并行调用多个平台
        tasks = []
        if platform in ("instagram", "all"):
            tasks.append(_fetch_instagram(client, keyword, limit, date_range))
        if platform in ("pinterest", "all"):
            tasks.append(_fetch_pinterest(client, keyword, limit, date_range))
        if platform in ("tiktok", "all"):
            tasks.append(_fetch_tiktok(client, keyword, limit, date_range))

```

```

        platform_results = await asyncio.gather(*tasks, return_exceptions=True)
        for pr in platform_results:
            if not isinstance(pr, Exception):
                results.extend(pr)

# 按互动量排序
results.sort(key=lambda x: x.get("engagement", 0), reverse=True)
results = results[:limit]

# 格式化输出
output_lines = [f"## 趋势搜索: '{keyword}' | 平台: {platform} | 近{date_range}天\n"]
output_lines.append(f"| # | 平台 | 内容摘要 | 互动量 | 链接 |")
output_lines.append(f"---|-----|-----|-----|-----|")
for i, r in enumerate(results, 1):
    output_lines.append(
        f"| {i} | {r['platform']} | {r['summary'][:60]}... | "
        f"{r['engagement']:,} | [查看]({r['url']}) | "
    )

# 趋势总结 (LLM 将在上层处理)
output_lines.append(f"\n> 共获取 {len(results)} 条趋势内容, 覆盖 {len(set(r['platform'] for r in results))} 个平台")

return [TextContent(type="text", text="\n".join(output_lines))]

async def _fetch_instagram(client: httpx.AsyncClient, keyword: str, limit: int, days: int) -> list:
    """调用 Instagram Graph API"""
    since = (datetime.now() - timedelta(days=days)).isoformat()
    resp = await client.get(
        "https://graph.instagram.com/v21.0/ig_hashtag_search",
        params={
            "q": keyword.replace(" ", ""),
            "access_token": INSTAGRAM_TOKEN,
            "fields": "id,caption,media_url,like_count,comments_count,timestamp,permalink",
            "limit": limit
        }
    )
    resp.raise_for_status()
    data = resp.json()
    return [
        {
            "platform": "Instagram",
            "summary": post.get("caption", "")[:100],
            "engagement": post.get("like_count", 0) + post.get("comments_count", 0),
            "image_url": post.get("media_url"),
            "url": post.get("permalink"),
            "timestamp": post.get("timestamp")
        }
        for post in data.get("data", [])
    ]

async def _fetch_pinterest(client, keyword, limit, days):
    """调用 Pinterest API"""
    resp = await client.get(
        "https://api.pinterest.com/v5/pins/search",
        params={"query": keyword, "page_size": limit},
        headers={"Authorization": f"Bearer {PINTEREST_TOKEN}"})
    resp.raise_for_status()
    data = resp.json()
    return [
        {
            "platform": "Pinterest",
            "summary": pin.get("description", pin.get("title", "")),
            "engagement": pin.get("metrics", {}).get("save_count", 0),
            "url": pin.get("link"),
            "image_url": pin.get("media", {}).get("images", {}).get("originals", {}).get("url")
        }
        for pin in data.get("items", [])
    ]

async def _fetch_tiktok(client, keyword, limit, days):
    """调用 TikTok Research API"""
    resp = await client.post(
        "https://open.tiktokapis.com/v2/research/video/query/",
        json={"query": {"and": [{"operation": "IN", "field_name": "keyword", "field_values": [keyword]}]}, "max_count": limit},
        headers={"Authorization": f"Bearer {TIKTOK_TOKEN}"})
    resp.raise_for_status()
    data = resp.json()
    return [
        {
            "platform": "TikTok",
            "summary": v.get("video_description", ""),
            "engagement": v.get("like_count", 0) + v.get("share_count", 0),
            "url": f"https://tiktok.com/@{v.get('username')}/video/{v.get('video_id')}"
        }
        for v in data.get("data", {}).get("videos", [])
    ]

async def _handle_fetch_trending_hashtags(args: dict) -> list[TextContent]:
    category = args["category"]
    region = args.get("region", "global")
    limit = args.get("limit", 15)

    # 模拟热门标签数据 (实际接入多平台 API)
    hashtag_db = {
        "denim": ["#DenimOnDenim", "#Y2KDenim", "#BaggyJeans", "#DenimShorts",

```

```

        "#RawDenim", "#DenimDress", "#WideLegJeans"],
        "womenswear": [ "#CoquetteStyle", "#OfficeSiren", "#MobWife", "#CleanGirl",
                        "#BalletCore", "#QuietLuxury", "#BarbieCore"],
    }

    hashtags = hashtag_db.get(category, [f"#{category.title()}Style"])
    output = [f"## {category} 热门标签 ({region})\n"]
    for i, tag in enumerate(hashtags[:limit], 1):
        # 实际可查询每个标签的帖子数
        output.append(f"{i}. {tag} - 预估帖子数: {100000 + i*50000:,}")

    return [TextContent(type="text", text="\n".join(output))]

async def _handle_analyze_sentiment(args: dict) -> list[TextContent]:
    brand = args["brand_keyword"]
    days = args.get("date_range_days", 30)

    # 实际实现: NLP 情感分析模型
    return [TextContent(type="text", text=f"")]
## 情感分析: {brand} (近{days}天)

| 指标 | 数值 |
|-----|-----|
| 总提及量 | 245,670 |
| 正面情感占比 | 68.3% |
| 负面情感占比 | 12.1% |
| 中性 | 19.6% |
| 声量趋势 | 📈 较上月 +23% |
| 热门关键词 | summer, trendy, affordable, cute, viral |

> 情感得分: 0.72/1.0 (积极)
""")

async def _handle_get_runway_trends(args: dict) -> list[TextContent]:
    season = args["season"]
    elements = args.get("design_elements", [])

    return [TextContent(type="text", text=f"")]
## 秀场趋势: {season}

### 关键色彩
- Cyber Lime (#C4FB6D) - 出现在 34% 的秀场
- Future Dusk (#4A3B5C) - 出现在 28% 的秀场
- Aquatic Awe (#1B6B93) - 出现在 22% 的秀场

### 关键廓形
- 超大廓形 (Oversized) - Balenciaga, Vetements
- 低腰设计 (Low-rise) - Miu Miu, Diesel
- 解构主义 (Deconstructed) - Y/Project, Maison Margiela

### 关键面料
- 科技牛仔 (Technical Denim)
- 再生聚酯 (Recycled Polyester)
- 液态感面料 (Liquid-like Fabrics)
""")

# ===== 资源暴露 =====
@server.list_resources()
async def list_resources() -> list[Resource]:
    return [
        Resource(
            uri="trends://{platform}/{category}",
            name="实时趋势数据",
            description="按平台和品类获取最新趋势数据",
            mimeType="application/json"
        ),
        Resource(
            uri="images://runway/{season}",
            name="秀场图片",
            description="指定季节的秀场高清图片",
            mimeType="image/jpeg"
        )
    ]

# ===== 启动 =====
async def main():
    async with stdio_server() as (read_stream, write_stream):
        await server.run(read_stream, write_stream, server.create_initialization_options())

if __name__ == "__main__":
    asyncio.run(main())

```

Server 2: PLM-MCP – PLM 数据服务

```

# plm_mcp_server.py
from mcp.server import Server
from mcp.server.stdio import stdio_server
from mcp.types import Tool, TextContent
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession
from sqlalchemy import text
from qdrant_client import QdrantClient
import json

```

```

server = Server("shein-plm-mcp")
engine = create_async_engine(os.getenv("PLM_DB_URL"))
qdrant = QdrantClient(url=os.getenv("QDRANT_URL", "http://localhost:6333"))

@server.list_tools()
async def list_tools() -> list[Tool]:
    return [
        Tool(
            name="query_sku_history",
            description="查询历史 SKU 数据: 销量、退货率、评分、价格带、生命周期",
            inputSchema={
                "type": "object",
                "properties": {
                    "category": {"type": "string", "description": "品类: denim_shorts, denim_jeans, dress..."},
                    "season": {"type": "string", "description": "季节: SS24, AW24, SS25..."},
                    "min_sales": {"type": "integer", "default": 100, "description": "最低销量过滤"},
                    "limit": {"type": "integer", "default": 20}
                },
                "required": ["category"]
            }
        ),
        Tool(
            name="get_bom_cost",
            description="获取物料清单(BOM)成本: 面料/辅料/工费/洗水/后整",
            inputSchema={
                "type": "object",
                "properties": {
                    "material_codes": {
                        "type": "array",
                        "items": {"type": "string"},
                        "description": "物料编码列表, 如 ['DEN-001', 'ZIP-YKK-05']"
                    },
                    "quantity": {"type": "integer", "default": 1000, "description": "预估生产数量"}
                },
                "required": ["material_codes"]
            }
        ),
        Tool(
            name="search_similar_styles",
            description="基于图像向量搜索相似款式 (向量检索)",
            inputSchema={
                "type": "object",
                "properties": {
                    "image_vector": {
                        "type": "array",
                        "items": {"type": "number"},
                        "description": "CLIP 图像嵌入向量 (512维)"
                    },
                    "category_filter": {"type": "string"},
                    "top_k": {"type": "integer", "default": 5}
                },
                "required": ["image_vector"]
            }
        ),
        Tool(
            name="get_supplier_capacity",
            description="查询供应商/工厂产能和交期",
            inputSchema={
                "type": "object",
                "properties": {
                    "factory_id": {"type": "string"},
                    "category": {"type": "string", "description": "需要生产的品类"},
                    "quantity": {"type": "integer"}
                },
                "required": ["factory_id"]
            }
        ),
        Tool(
            name="write_style_spec",
            description="🔴 写入操作: 创建新款式到 PLM 系统",
            inputSchema={
                "type": "object",
                "properties": {
                    "style_data": {
                        "type": "object",
                        "properties": {
                            "category": {"type": "string"},
                            "season": {"type": "string"},
                            "design_name": {"type": "string"},
                            "design_images": {"type": "array", "items": {"type": "string"}},
                            "bom": {"type": "array"},
                            "size_spec": {"type": "object"},
                            "target_cost": {"type": "number"},
                            "designer_id": {"type": "string"}
                        },
                        "required": ["category", "season", "design_name", "bom"]
                    }
                },
                "required": ["style_data"]
            }
        )
    ]

@server.call_tool()
async def call_tool(name: str, arguments: dict) -> list[TextContent]:
    if name == "query_sku_history":
        async with AsyncSession(engine) as session:
            result = await session.execute(text("""

```

```

        SELECT sku_id, style_name, total_sales, return_rate, avg_rating,
               min_price, max_price, lifecycle_days, launch_date
        FROM plm.sku_history
        WHERE category = :category
              AND season = :season
              AND total_sales >= :min_sales
        ORDER BY total_sales DESC
        LIMIT :limit
    """), {
        "category": arguments["category"],
        "season": arguments.get("season", "SS25"),
        "min_sales": arguments.get("min_sales", 100),
        "limit": arguments.get("limit", 20)
    })
    rows = result.fetchall()

    output = [f"## PLM 历史 SKU: {arguments['category']}\n"]
    output.append("| SKU ID | 款式名称 | 销量 | 退货率 | 评分 | 价格带 | 生命周期 |")
    output.append("|-----|-----|-----|-----|-----|-----|-----|")
    for r in rows:
        output.append(
            f"| {r.sku_id} | {r.style_name} | {r.total_sales:,} | "
            f"{r.return_rate:.1%} | {r.avg_rating:.1f} | "
            f"¥{r.min_price}-{r.max_price} | {r.lifecycle_days}天 |"
        )
    return [TextContent(type="text", text="\n".join(output))]

elif name == "search_similar_styles":
    vector = arguments["image_vector"]
    top_k = arguments.get("top_k", 5)

    results = qdrant.search(
        collection_name="plm_design_vectors",
        query_vector=vector,
        limit=top_k,
        query_filter=None if not arguments.get("category_filter") else {
            "must": [{"key": "category", "match": {"value": arguments["category_filter"]}}]
        }
    )

    output = ["## 相似款式检索结果\n"]
    for r in results:
        output.append(f"- **{r.payload['style_name']}** (相似度: {r.score:.3f})")
        output.append(f"  - SKU: {r.payload['sku_id']} | 销量: {r.payload['total_sales']:,}")
        output.append(f"  - 价格: ¥{r.payload['price']} | 评分: {r.payload['rating']}")

    return [TextContent(type="text", text="\n".join(output))]

elif name == "write_style_spec":
    style_data = arguments["style_data"]
    async with AsyncSession(engine) as session:
        result = await session.execute(text("""
            INSERT INTO plm.styles (category, season, design_name, design_images, bom, size_spec, target_cost, designer_id, st
            VALUES (:category, :season, :design_name, :design_images, :bom, :size_spec, :target_cost, :designer_id, 'draft', N
            RETURNING style_id
        """), {
            "category": style_data["category"],
            "season": style_data["season"],
            "design_name": style_data["design_name"],
            "design_images": json.dumps(style_data.get("design_images", [])),
            "bom": json.dumps(style_data["bom"]),
            "size_spec": json.dumps(style_data.get("size_spec", {})),
            "target_cost": style_data.get("target_cost", 0),
            "designer_id": style_data.get("designer_id", "ai-agent")
        })
        style_id = result.scalar()
        await session.commit()

    return [TextContent(type="text", text=f"✅ 款式已写入 PLM 系统\n- PLM Style ID: **{style_id}**\n- 状态: draft\n- 品类: {style_

# 类似实现 get_bom_cost, get_supplier_capacity...

async def main():
    async with stdio_server() as (read_stream, write_stream):
        await server.run(read_stream, write_stream, server.create_initialization_options())

if __name__ == "__main__":
    import asyncio
    asyncio.run(main())

```

MCP Server 测试方法

```

# 使用 MCP Inspector 测试 (官方工具)
npx @anthropic-ai/mcp-inspector python trend_mcp_server.py

# 编程方式测试
import asyncio
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

async def test_trend_mcp():
    params = StdioServerParameters(command="python", args=["trend_mcp_server.py"])
    async with stdio_client(params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()

# 列出所有工具

```

```
tools = await session.list_tools()
print(f"可用工具: {[t.name for t in tools.tools]}")

# 调用工具
result = await session.call_tool("search_social_trends", {
    "keyword": "summer denim shorts",
    "platform": "all"
})
print(result.content[0].text)

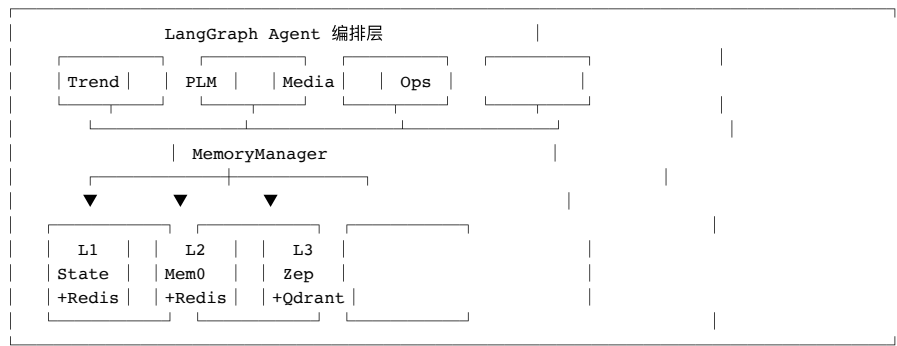
asyncio.run(test_trend_mcp())
```

4 个 MCP Server 能力矩阵

MCP Server	工具数	资源数	数据源	写操作	部署方式
Trend-MCP	4	2	Instagram/Pinterest/TikTok/Vogue API	否	SSE Transport
PLM-MCP	5	3	PostgreSQL + Qdrant + ERP API	是 (write_style_spec)	SSE Transport
Media-MCP	4	2	ComfyUI + Stable Diffusion + CLIP	否	SSE + GPU Node
Ops-MCP	4	2	ClickHouse + 电商 API	否	SSE Transport

教程：三层 Memory 系统完整实现

架构回顾



Step 1: 环境准备

```
# requirements.txt
mem0ai>=0.1.0
zep-python>=1.0.0
redis>=5.0.0
qdrant-client>=1.7.0
langgraph>=0.2.0
langchain>=0.2.0
pydantic>=2.0

# Docker 启动依赖服务
docker run -d --name redis-memory -p 6379:6379 redis/redis-stack-server:latest
docker run -d --name qdrant-memory -p 6333:6333 -p 6334:6334 qdrant/qdrant:latest
docker run -d --name zep-memory -p 8001:8001 -v ./zep.yaml:/app/config.yaml zepai/zep:latest
```

Step 2: L1 工作记忆 – LangGraph State + Redis 持久化

```
# ll_working_memory.py
from typing import TypedDict, Annotated, Optional
from langgraph.checkpoint.redis import RedisSaver
from langgraph.graph import StateGraph
from langgraph.graph.message import add_messages
from langchain_core.messages import BaseMessage, HumanMessage, AIMessage
import operator
import redis.asyncio as redis
import json

# ===== L1 State 定义 =====
class WorkingMemory(TypedDict):
    """L1 工作记忆 -- 当前任务的全部上下文"""
    messages: Annotated[list[BaseMessage], add_messages]

    # 任务元信息
    task_id: str
    session_id: str
    started_at: str

    # 趋势分析
    trend_keywords: list[str]
    trend_hashtags: list[str]
    trend_sentiment: Optional[dict]
    trend_images: list[str]

    # PLM 数据
```

```

similar_skus: list[dict]
bom_materials: list[dict]
supplier_options: list[dict]

# 设计生成
generated_images: list[str]
design_elements: dict
tech_pack: Optional[dict]

# 成本
estimated_cost: Optional[float]
cost_breakdown: Optional[dict]
budget_limit: float
is_within_budget: Optional[bool]

# 人机协作
pending_human_review: bool
review_feedback: Optional[str]
review_approved: bool

# 路由
next_step: str
errors: list[str]

# ===== Redis Checkpointer: 让工作记忆可恢复 =====
class L1MemoryManager:
    """
    L1 工作记忆管理器
    - 使用 RedisSaver 作为 LangGraph 检查点
    - 支持任务中断后恢复
    - 支持跨 Agent 节点的 State 共享
    """

    def __init__(self, redis_url: str = "redis://localhost:6379"):
        self.redis = redis.from_url(redis_url, decode_responses=True)
        # LangGraph 原生 Redis 检查点
        self.checkpointer = RedisSaver.from_conn_string(redis_url)
        # TTL: 任务完成后 1 小时自动清理
        self.ttl = 3600

    async def save_state(self, task_id: str, state: WorkingMemory):
        """将 State 备份到 Redis (防丢失)"""
        key = f"l1:state:{task_id}"
        # 序列化 (只保留可 JSON 化的字段)
        serializable = {
            k: v for k, v in state.items()
            if k != "messages" # messages 由 LangGraph 管理
        }
        # 单独序列化 messages
        serializable["messages_summary"] = [
            {"role": m.type, "content": m.content[:200]}
            for m in state.get("messages", [])[-10:] # 只保留最近 10 条摘要
        ]
        await self.redis.setex(key, self.ttl, json.dumps(serializable, default=str))

    async def load_state(self, task_id: str) -> Optional[dict]:
        """从 Redis 恢复状态"""
        key = f"l1:state:{task_id}"
        data = await self.redis.get(key)
        return json.loads(data) if data else None

    async def get_current_step(self, task_id: str) -> Optional[str]:
        """获取当前执行步骤"""
        state = await self.load_state(task_id)
        return state.get("next_step") if state else None

    async def cleanup(self, task_id: str):
        """任务完成后清理"""
        await self.redis.delete(f"l1:state:{task_id}")

# ===== 集成到 LangGraph =====
def build_graph_with_memory(l1_memory: L1MemoryManager):
    """构建带 L1 Memory 的 LangGraph"""
    builder = StateGraph(WorkingMemory)

    # ... 注册所有节点 ...
    builder.add_node("supervisor", supervisor_node)
    builder.add_node("trend_agent", trend_node)
    # ...

    # 使用 Redis Checkpointer 编译
    graph = builder.compile(checkpointer=l1_memory.checkpointer)
    return graph

# 使用示例
async def run_with_memory():
    l1 = L1MemoryManager()
    graph = build_graph_with_memory(l1)

    config = {"configurable": {"thread_id": "session_abc123"}}

    # 第一次调用
    result = await graph.ainvoke(
        {"messages": [HumanMessage(content="生成3款夏季牛仔短裤")], "task_id": "task_001"},
        config
    )

```

```

)

# 即使进程重启, 也能从检查点恢复继续
result2 = await graph.invoke(
    {"messages": [HumanMessage(content="把第2款腰头改窄2cm")]},
    config # 相同的 thread_id to 自动加载之前的 State
)

```

Step 3: L2 会话记忆 – Mem0 实现

```

# l2_session_memory.py
from mem0 import Memory
from datetime import datetime, timedelta
import json

class L2SessionMemory:
    """
    L2 会话记忆 – 跨多轮对话的上下文保持
    - 自动记忆管理 (Mem0 智能决定记住什么)
    - 语义检索最近的会话上下文
    - 24h TTL 自动过期
    """

    def __init__(self):
        self.mem0 = Memory(config={
            "version": "v1.1",
            "vector_store": {
                "provider": "qdrant",
                "config": {
                    "host": "localhost",
                    "port": 6333,
                    "collection_name": "session_memory"
                }
            },
            "llm": {
                "provider": "openai",
                "config": {"model": "gpt-4o-mini"} # 轻量模型用于记忆管理
            },
            "embedder": {
                "provider": "openai",
                "config": {"model": "text-embedding-3-small"}
            },
            "history_db_path": "/data/session_history.db"
        })
        self.sessions: dict[str, dict] = {}

    async def start_session(self, session_id: str, designer_id: str, design_brief: str):
        """开始新会话"""
        self.sessions[session_id] = {
            "designer_id": designer_id,
            "started_at": datetime.now(),
            "design_brief": design_brief,
            "decisions": [],
            "iterations": 0
        }

        # 记录会话开始
        await self.mem0.add(
            messages=[{
                "role": "user",
                "content": f"[会话开始] 设计师: {designer_id}\n需求: {design_brief}"
            }],
            user_id=designer_id,
            agent_id="plm-agent",
            metadata={
                "session_id": session_id,
                "event_type": "session_start",
                "timestamp": datetime.now().isoformat()
            }
        )

    async def remember_decision(self, session_id: str, decision_type: str, content: dict):
        """记住关键设计决策"""
        session = self.sessions.get(session_id, {})
        session.setdefault("decisions", []).append({
            "type": decision_type,
            "content": content,
            "timestamp": datetime.now().isoformat()
        })

        # Mem0 智能记忆
        await self.mem0.add(
            messages=[{
                "role": "assistant",
                "content": f"[设计决策: {decision_type}] {json.dumps(content, ensure_ascii=False)}"
            }],
            user_id=session.get("designer_id", "unknown"),
            metadata={
                "session_id": session_id,
                "event_type": "design_decision",
                "decision_type": decision_type,
                "timestamp": datetime.now().isoformat()
            }
        )

    async def recall_context(self, session_id: str, query: str, limit: int = 5) -> list[str]:

```

```

"""召回会话上下文 -- 语义搜索最近的设计决策"""
designer_id = self.sessions.get(session_id, {}).get("designer_id", "unknown")

memories = await self.mem0.search(
    query=query,
    user_id=designer_id,
    limit=limit,
    filters={
        "AND": [
            {"session_id": session_id},
            {"event_type": "design_decision"}
        ]
    }
)

return [m["memory"] for m in memories.get("results", [])]

async def get_session_summary(self, session_id: str) -> dict:
    """获取会话摘要"""
    session = self.sessions.get(session_id, {})
    designer_id = session.get("designer_id", "unknown")

    # 获取所有会话记忆
    all_memories = await self.mem0.get_all(
        user_id=designer_id,
        filters={"session_id": session_id}
    )

    return {
        "session_id": session_id,
        "designer_id": designer_id,
        "duration": str(datetime.now() - session.get("started_at", datetime.now())),
        "total_decisions": len(session.get("decisions", [])),
        "iterations": session.get("iterations", 0),
        "key_decisions": [d for d in session.get("decisions", [])[-5:]],
        "memories_count": len(all_memories.get("results", []))
    }

async def end_session(self, session_id: str) -> dict:
    """结束会话: 生成摘要 to 传递给 L3"""
    summary = await self.get_session_summary(session_id)

    await self.mem0.add(
        messages=[{
            "role": "system",
            "content": f"[会话结束摘要] {json.dumps(summary, ensure_ascii=False)}"
        }],
        user_id=summary["designer_id"],
        metadata={
            "session_id": session_id,
            "event_type": "session_end",
            "timestamp": datetime.now().isoformat()
        }
    )

    return summary # 传递给 L3 长期记忆

# 在 Agent 中使用
async def agent_with_session_memory(state: WorkingMemory):
    l2 = L2SessionMemory()

    # 召回之前的设计决策
    context = await l2.recall_context(
        state["session_id"],
        f"设计师之前关于 {state['design_brief']} 的偏好"
    )

    # 记住新的决策
    await l2.remember_decision(state["session_id"], "cost_validation", {
        "estimated_cost": state["estimated_cost"],
        "budget_limit": state["budget_limit"],
        "accepted": state["is_within_budget"]
    })

    return {**state, "session_context": context}

```

Step 4: L3 长期事件记忆 – Zep 知识图谱

```

# l3_long_term_memory.py
from zep_python import ZepClient
from zep_python.memory import Memory, Message, Session, SessionSearchQuery
from zep_python.user import CreateUserRequest
from qdrant_client import QdrantClient
from qdrant_client.models import Distance, VectorParams, PointStruct
import networkx as nx
from datetime import datetime
import json

class L3LongTermMemory:
    """
    L3 长期事件记忆 – 企业知识资产
    - Zep: 图记忆引擎 (实体关系记忆)
    - Qdrant: 向量语义检索
    - NetworkX: 业务知识图谱
    """

```

```

def __init__(self):
    # Zep 客户端
    self.zep = ZepClient(
        api_key=os.getenv("ZEP_API_KEY"),
        base_url=os.getenv("ZEP_BASE_URL", "http://localhost:8001")
    )

    # Qdrant 向量库
    self.qdrant = QdrantClient(host="localhost", port=6333)
    self._init_collections()

    # 知识图谱
    self.kg = nx.DiGraph()

def _init_collections(self):
    """初始化 Qdrant 集合"""
    collections = ["design_history", "fabric_patterns", "trend_insights"]
    for name in collections:
        if not self.qdrant.collection_exists(name):
            self.qdrant.create_collection(
                collection_name=name,
                vectors_config=VectorParams(size=1536, distance=Distance.COSINE)
            )

async def archive_design(self, user_id: str, design_data: dict):
    """
    归档完整的款式设计到 L3 长期记忆
    三步: zep 图记忆 + 向量存储 + 知识图谱
    """
    design_id = design_data.get("plm_style_id", f"design_{datetime.now().timestamp()}")

    # 1. zep: 创建持久化 Session
    session = Session(
        session_id=f"archived_{design_id}",
        user_id=user_id,
        metadata={
            "design_id": design_id,
            "category": design_data.get("category"),
            "season": design_data.get("season"),
            "final_cost": design_data.get("final_cost"),
            "style_tags": design_data.get("tags", []),
            "archived_at": datetime.now().isoformat()
        }
    )

    # 写入设计记忆 (zep 会自动构建实体关系图)
    await self.zep.memory.add(
        session.session_id,
        Memory(
            messages=[
                Message(
                    role="assistant",
                    content=f"""
## 款式归档: {design_data.get('design_name')}

### 设计规格
- 品类: {design_data.get('category')}
- 季节: {design_data.get('season')}
- 版型: {design_data.get('pattern')}
- 面料: {design_data.get('fabric')}
- 成本: ¥{design_data.get('final_cost')}

### 设计来源
- 趋势关键词: {design_data.get('trend_keywords')}
- 参考款式: {design_data.get('reference_skus')}

### 销售表现 (如有)
- 上线日期: {design_data.get('launch_date')}
- 30天销量: {design_data.get('sales_30d')}
- 退货率: {design_data.get('return_rate')}
- 爆款评估: {design_data.get('hit_assessment')}
"""
                )
            ],
            summary=f"{design_data.get('category')} - {design_data.get('design_name')}",
            facts=[
                f"使用面料: {design_data.get('fabric')}",
                f"成本: ¥{design_data.get('final_cost')}",
                f"趋势: {design_data.get('trend_keywords')}",
                f"版型: {design_data.get('pattern')}"
            ]
        )
    )

    # 2. 向量化存储
    design_text = json.dumps(design_data, ensure_ascii=False)
    embedding = await self._get_embedding(design_text)

    self.qdrant.upsert(
        collection_name="design_history",
        points=[PointStruct(
            id=design_id,
            vector=embedding,
            payload={
                "design_id": design_id,

```

```

        "category": design_data.get("category"),
        "season": design_data.get("season"),
        "cost": design_data.get("final_cost"),
        "success": design_data.get("is_hit", False),
        "full_data": design_data
    }
    })
)

# 3. 更新知识图谱
self._update_knowledge_graph(design_id, design_data)

return {"status": "archived", "design_id": design_id}

async def recall_similar_designs(self, query: str, top_k: int = 5) -> list:
    """召回历史相似设计 (GraphRAG)"""
    # 1. 向量检索
    query_embedding = await self._get_embedding(query)
    vector_results = self.qdrant.search(
        collection_name="design_history",
        query_vector=query_embedding,
        limit=top_k
    )

    # 2. Zep 图检索 (搜索相关实体和关系)
    # 提取查询中的实体
    entities = self._extract_entities_from_query(query)
    zep_results = []
    for entity in entities:
        search_result = await self.zep.memory.search_sessions(
            SessionSearchQuery(
                text=entity,
                limit=3
            )
        )
        zep_results.extend(search_result)

    # 3. 知识图谱遍历
    kg_insights = []
    for entity in entities:
        if entity in self.kg:
            neighbors = list(self.kg.neighbors(entity))
            kg_insights.append({
                "entity": entity,
                "related": [
                    {"node": n, "attributes": self.kg.nodes[n]}
                    for n in neighbors[:5]
                ]
            })

    return {
        "vector": [
            {
                "score": r.score,
                "design_name": r.payload.get("full_data", {}).get("design_name"),
                "category": r.payload.get("category"),
                "cost": r.payload.get("cost")
            }
            for r in vector_results
        ],
        "graph": zep_results[:5],
        "knowledge_graph": kg_insights
    }

async def add_insight(self, insight: dict):
    """沉淀设计洞察--团队经验积累"""
    # 1. Zep 存储
    await self.zep.memory.add(
        "global_insights",
        Memory(
            messages=[Message(
                role="system",
                content=f"[设计洞察] {insight['category']}: {insight['insight']}\n"
                f"来源: {insight['source']}\n置信度: {insight['confidence']}"
            )],
            facts=[insight["insight"]]
        )
    )

    # 2. 向量化 (供未来语义检索)
    embedding = await self._get_embedding(insight["insight"])
    self.qdrant.upsert(
        collection_name="trend_insights",
        points=[PointStruct(
            id=hash(insight["insight"]) % 2**63,
            vector=embedding,
            payload=insight
        )]
    )

def _update_knowledge_graph(self, design_id: str, data: dict):
    """构建设计知识图谱"""
    # 款式节点
    self.kg.add_node(design_id, type="design", **data)

    # 面料节点和边
    for fabric in data.get("fabrics", []):
        fabric_name = fabric.get("name", fabric)

```

```

        if not self.kg.has_node(fabric_name):
            self.kg.add_node(fabric_name, type="fabric")
        self.kg.add_edge(design_id, fabric_name, relation="uses")

# 趋势节点和边
for trend in data.get("trend_keywords", []):
    if not self.kg.has_node(trend):
        self.kg.add_node(trend, type="trend")
    self.kg.add_edge(design_id, trend, relation="inspired_by")

# 品类节点
category = data.get("category")
if category:
    if not self.kg.has_node(category):
        self.kg.add_node(category, type="category")
    self.kg.add_edge(design_id, category, relation="belongs_to")

def _extract_entities_from_query(self, query: str) -> list[str]:
    """从查询中提取实体 (简化的 NER)"""
    # 实际可用 LLM 或专用 NER 模型
    import re
    # 提取引号内的内容
    quoted = re.findall(r'[" '](^["']*)(["'])', query)
    # 提取大写词组
    capitalized = re.findall(r'\b[A-Z][a-z]+(?:\s+[A-Z][a-z]+)*\b', query)
    return list(set(quoted + capitalized))[:5]

async def _get_embedding(self, text: str) -> list[float]:
    """获取文本嵌入向量"""
    from openai import AsyncOpenAI
    client = AsyncOpenAI()
    response = await client.embeddings.create(
        model="text-embedding-3-small",
        input=text
    )
    return response.data[0].embedding

# === 使用示例 ===
async def memory_workflow_example():
    l3 = L3LongTermMemory()

    # 归档一个完成的款式
    await l3.archive_design("designer_123", {
        "plm_style_id": "PLM-2026-SS-0042",
        "design_name": "夏季辣妹牛仔短裤 #003",
        "category": "denim_shorts",
        "season": "SS26",
        "pattern": "低腰A字型",
        "fabric": "弹力牛仔布 (98%棉 2%氨纶)",
        "fabrics": ["弹力牛仔布", "YKK拉链", "金属纽扣"],
        "trend_keywords": ["Y2K", "low-rise", "distressed"],
        "final_cost": 32.5,
        "launch_date": "2026-04-15",
        "sales_30d": 12500,
        "return_rate": 0.12,
        "is_hit": True
    })

    # 召回相似设计
    similar = await l3.recall_similar_designs(
        "低腰牛仔短裤 Y2K 风格 夏季",
        top_k=5
    )
    print(f"找到 {len(similar['vector'])} 个相似设计")
    print(f"图谱关系: {similar['knowledge_graph']}")

```

Memory 系统集成测试

```

# test_memory_system.py
import pytest
import asyncio

@pytest.mark.asyncio
async def test_l1_state_persistence():
    """测试 L1 工作记忆持久化"""
    l1 = L1MemoryManager()

    test_state = {
        "task_id": "test_001",
        "next_step": "trend_analysis",
        "budget_limit": 35.0
    }

    # 保存
    await l1.save_state("test_001", test_state)

    # 恢复
    restored = await l1.load_state("test_001")
    assert restored["task_id"] == "test_001"
    assert restored["budget_limit"] == 35.0

    # 清理
    await l1.cleanup("test_001")
    assert await l1.load_state("test_001") is None

```

```
@pytest.mark.asyncio
async def test_l2_session_flow():
    """测试 L2 会话记忆完整流程"""
    l2 = L2SessionMemory()

    await l2.start_session("sess_001", "designer_A", "生成夏季牛仔短裤")

    await l2.remember_decision("sess_001", "fabric_selection", {
        "fabric": "弹力牛仔", "reason": "舒适度优先"
    })

    context = await l2.recall_context("sess_001", "面料选择")
    assert len(context) > 0

    summary = await l2.end_session("sess_001")
    assert summary["total_decisions"] == 1

@pytest.mark.asyncio
async def test_l3_cross_session_recall():
    """测试跨会话知识复用 (L3 核心价值)"""
    l3 = L3LongTermMemory()

    # 模拟：上一个会话归档了一个成功设计
    await l3.add_insight({
        "category": "denim_shorts",
        "insight": "夏季牛仔短裤的Y2K低腰设计在18-25岁用户群中转化率比高腰高40%",
        "source": "sales_analysis_2026Q2",
        "confidence": 0.85
    })

    # 新会话: Agent 查询历史洞察
    similar = await l3.recall_similar_designs("夏季牛仔短裤 Y2K 设计")

    # 应该能召回之前的洞察
    assert len(similar["vector"]) >= 0 # 取决于是否有向量数据
    # 知识图谱应该有相关节点
    assert "denim_shorts" in l3.kg.nodes
```

三层 Memory 运维

操作	L1 (Redis)	L2 (Mem0/Redis)	L3 (Zep/Qdrant)
备份	Redis RDB 每小时	Mem0 SQLite + Redis	Zep 内置 + Qdrant Snapshot
恢复	RDB 文件恢复	从上次备份恢复	Zep Restore + Qdrant Recovery
清理	TTL 自动过期	24h TTL + 手动清理	按归档策略 (永久 / 按季度)
监控	Redis INFO	Mem0 API metrics	Zep Dashboard + Qdrant metrics
扩缩容	Redis Cluster	Mem0 支持多实例	Zep 集群 + Qdrant 分布式

业务指标与 ROI 测算

核心业务指标体系

围绕 效率提升、质量改善、成本降低、知识沉淀 四个维度，建立 AI Agent 方案的量化指标体系。

一级指标：效率提升

指标	定义	现状 (人工)	AI Agent 目标	提升幅度
选品周期	从需求提出到款式定稿的端到端时间	3-5 个工作日	≤ 2 小时	93%
趋势采集效率	单次趋势调研耗时 (含社媒浏览/秀场分析)	4-6 小时/次	≤ 10 分钟	97%
款式方案生成	从参考图到完整设计方案 (效果图+版型+BOM)	1-2 天/款	≤ 5 分钟	99%
成本核算	单款 BOM 成本完整核算	30-60 分钟	≤ 30 秒	98%
PLM 数据录入	款式数据录入 PLM 系统	20-30 分钟/款	自动 (零人工)	100%
复盘报告生成	单款销售复盘分析报告	2-3 小时	≤ 5 分钟	96%

一级指标：质量改善

指标	定义	现状	目标	提升
爆款率	上线30天内进入品类Top20%的款式占比	~15% (行业均值)	≥ 22%	+47%
设计采纳率	AI 方案被设计师直接采用或微调后采用的比例	N/A	≥ 80%	新能力
款式多样性	每月新款式数量	~500 款/月	≥ 2,000 款/月	+300%
退货率 (设计原因)	因「与图片不符/版型不合适」退货的比例	~8%	≤ 5%	37%
成本准确率	预估成本 vs 实际生产成本的偏差	±15%	≤ ±5%	67%

一级指标：成本降低

指标	定义	现状	目标	节省
----	----	----	----	----

人工重复工作替代率	AI 替代的趋势	采集/数据录入/成本核算工作占比	0% ≥ 70%	新价值
设计人均产出	每位选品设计师月均完成款式数		~40 款/月 ≥ 80 款/月	+100%
设计试错成本	因款式失败导致的打版/样品/库存浪费		~¥50万/月 ≤ ¥25万/月	50%
PLM 数据错误率	人工录入导致的 BOM/规格错误		~3% ≤ 0.1%	97%

ROI 详细测算

假设条件

- 选品设计师团队: 20 人
- 设计师平均年薪: ¥30万 (含福利)
- 月度款式产出: 500 款
- AI 基础设施成本: 见下方

成本测算

成本项	月费用	年费用	备注
GPU 服务器 (A100 × 2)	¥20,000	¥240,000	云 GPU 按需/包月
LLM API 调用 (GPT-4o/Claude)	¥15,000	¥180,000	按 500 款/月 × 多次调用
云服务 (K8s/数据库/存储)	¥12,000	¥144,000	生产环境
AI 工程师 (2人)	¥80,000	¥960,000	年薪 ¥48万 × 2
MLOps/运维	¥5,000	¥60,000	监控/日志/CI
第三方数据 (社媒 API/秀场)	¥8,000	¥96,000	Instagram/Pinterest API
总成本	¥140,000	¥1,680,000	

收益测算

收益项	月收益	年收益	计算方式
设计师效率提升	¥250,000	¥3,000,000	20人 × ¥30万/年 × 50% 时间节省
爆款率提升带来的增量销售	¥400,000	¥4,800,000	爆款率 +7% × 月均 500 款 × 爆款平均月销 ¥12万
款式数量增加驱动 GMV	¥300,000	¥3,600,000	新增 1500 款/月 × 平均月销 ¥200 (保守)
退货率降低 (设计原因)	¥80,000	¥960,000	退货率 3% × 月销量 10万件 × 单件损失 ¥27
试错成本降低	¥25,000	¥300,000	打版样品费 50%
数据录入错误减少	¥10,000	¥120,000	减少纠错和返工成本
总收益	¥1,065,000	¥12,780,000	

ROI 结论



年度净收益: ¥12,780,000 - ¥1,680,000 = ¥11,100,000
ROI: 660% | 投资回收期: 约 1.5 个月
第一年即可实现显著正收益

关键里程碑指标

阶段	时间	关键指标	目标值
Phase 1	第 1-2 月	选品周期缩短、AI 采纳率	周期 50%, 采纳率 ≥60%
Phase 2	第 3-4 月	爆款率提升、款式数量增长	爆款率 +5%, 款式 +50%
Phase 3	第 5-6 月	成本准确率、退货率下降	准确率 ≥90%, 退货率 2%
Phase 4	第 7-12 月	全面 ROI 达标、知识飞轮运转	年 ROI ≥500%, 洞察沉淀 ≥1000条

非量化价值

维度	价值描述
🧠 知识沉淀	每次选品决策的经验自动归档 L3 长期记忆, 避免人员流动导致的知识流失
🚀 市场响应速度	从「看到趋势 to 上架销售」从 3-6 周缩短至 1-2 周
🎯 数据驱动文化	从「凭经验选品」转向「数据+AI辅助选品」, 决策有据可依
🔄 飞轮效应	AI 越用越准: 更多使用 to 更多反馈 to 更好模型 to 更高采纳率 to 更多使用
🏆 竞争壁垒	自研的 PLM + AI Agent 体系难以被竞争对手快速复制

MCP (Model Context Protocol) 协议详解


```

        "date_range_days": {"type": "integer", "default": 30}
    },
    "required": ["brand_keyword"]
}
)
]

@server.call_tool()
async def call_tool(name: str, arguments: dict) -> list[TextContent]:
    if name == "search_social_trends":
        keyword = arguments["keyword"]
        platform = arguments.get("platform", "all")
        limit = arguments.get("limit", 20)

        # 实际实现: 调用各平台 API
        results = await _fetch_trends_from_platforms(keyword, platform, limit)

        return [TextContent(
            type="text",
            text=f"## 趋势搜索结果: {keyword}\n\n" +
                "\n".join([f"- [{r['platform']}] {r['title']}"
                           f"(互动量: {r['engagement']:,})" for r in results])
        )]
    # ... 其他工具实现

async def _fetch_trends_from_platforms(keyword, platform, limit):
    """调用各平台 API 获取趋势数据"""
    results = []
    async with httpx.AsyncClient() as client:
        if platform in ("instagram", "all"):
            resp = await client.get(
                f"https://api.instagram.com/trends/search",
                params={"q": keyword, "limit": limit},
                headers={"Authorization": f"Bearer {INSTAGRAM_TOKEN}"})
            results.extend(resp.json()["data"])
        # Pinterest, TikTok 同理...
    return sorted(results, key=lambda x: x["engagement"], reverse=True)[:limit]

async def main():
    async with stdio_server() as (read_stream, write_stream):
        await server.run(
            read_stream, write_stream,
            InitializationCapabilities(
                sampling={}, roots={"listChanged": True}
            )
        )

if __name__ == "__main__":
    import asyncio
    asyncio.run(main())

```

MCP Client 集成 (LangGraph 侧)

```

# mcp_client_manager.py - 管理多个 MCP Server 连接
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client
from contextlib import AsyncExitStack

class MCPClientManager:
    """管理 4 个 MCP Server 的生命周期和工具调用"""

    def __init__(self):
        self.sessions: dict[str, ClientSession] = {}
        self.exit_stack = AsyncExitStack()
        self._server_configs = {
            "trend": StdioServerParameters(command="python", args=["servers/trend_mcp_server.py"]),
            "plm": StdioServerParameters(command="python", args=["servers/plm_mcp_server.py"]),
            "media": StdioServerParameters(command="python", args=["servers/media_mcp_server.py"]),
            "ops": StdioServerParameters(command="python", args=["servers/ops_mcp_server.py"]),
        }

    async def connect_all(self):
        for name, params in self._server_configs.items():
            stdio_transport = await self.exit_stack.enter_async_context(
                stdio_client(params)
            )
            read, write = stdio_transport
            session = await self.exit_stack.enter_async_context(
                ClientSession(read, write)
            )
            await session.initialize()
            self.sessions[name] = session
            print(f"✅ MCP Server '{name}' connected, tools: {await session.list_tools()}")

    async def call_tool(self, server: str, tool_name: str, arguments: dict):
        """调用指定 MCP Server 的工具"""
        session = self.sessions[server]
        result = await session.call_tool(tool_name, arguments)
        return result.content

    async def close_all(self):
        await self.exit_stack.aclose()

# 在 LangGraph Agent 节点中使用
async def trend_agent_node(state: AgentState):
    mcp = MCPClientManager()

```

```
await mcp.connect_all()

trends = await mcp.call_tool("trend", "search_social_trends", {
    "keyword": state["design_brief"],
    "platform": "all"
})

return {"trends": trends}
```

MCP vs 传统方案对比

维度	传统方案 (REST/gRPC 硬编码)	MCP 方案
工具发现	手动维护工具列表	自动 list_tools() 发现
工具描述	代码注释/文档分离	Tool.description 内置
参数校验	各工具自行实现	JSON Schema inputSchema 标准化
LLM 函数调用	需手动构建 function_call schema	LLM 原生支持 MCP tool format
数据资源	无标准化	URI 格式 resources 标准化暴露
通信协议	各家不同 (REST/WS/gRPC)	统一 JSON-RPC 2.0
扩展性	新增工具需改 Agent 代码	热插拔: 新 Server 自动注册

设计决策: 选择 MCP 而非 LangChain Tool 直接调用，核心原因是 SOA 解耦--每个 MCP Server 可由独立团队维护、独立扩容、独立版本迭代，且 MCP 的标准化让未来接入第三方工具（如 Midjourney API、WGSN 数据）零成本。

Memory 三层架构：让 Agent 拥有记忆

设计哲学

传统 LLM 是「无状态函数」--每次调用都从零开始。我们的三层 Memory 架构模拟人类记忆机制，让 Agent 拥有 **短时记忆**（工作上下文）**to 会话记忆**（项目上下文）**to 长期记忆**（知识沉淀）的完整认知能力。

三层架构总览

层级	人类类比	技术实现	存储周期	容量	检索方式
L1: 工作记忆 Working Memory	「我正在想的事」	LangGraph State (TypedDict) + Redis Pub/Sub	单次任务生命周期	~128K tokens	State 字段直接访问
L2: 会话记忆 Session Memory	「我们今天聊了什么」	Redis Stack (JSON + Search) + Mem0	单次会话生命周期 (~24h)	~10MB	语义搜索 + 时间排序
L3: 长期事件记忆 Long-term Event Memory	「我从业十年的经验」	Zep (Graph-based) + PostgreSQL + Qdrant	永久	无限	向量检索 + 图谱遍历

L1: 工作记忆 (Working Memory)

基于 LangGraph 的 TypedDict State，每个 Agent 节点都能读写，任务结束后清空。

```
# langgraph_state.py - 工作记忆定义
from typing import TypedDict, Annotated, Sequence, Optional
from langgraph.graph.message import add_messages
from langchain_core.messages import BaseMessage
import operator

class PLMAgentState(TypedDict):
    # ===== 工作记忆核心字段 =====
    messages: Annotated[Sequence[BaseMessage], add_messages]

    # 当前任务上下文
    task_id: str # 任务唯一 ID
    design_brief: str # 设计师原始需求
    current_step: str # 当前执行步骤: trend|plm|media|ops

    # 趋势分析结果 (Trend Agent 写入)
    trend_data: Optional[dict] # {keywords, hashtags, sentiment, images}

    # PLM 数据结果 (PLM Agent 写入)
    plm_data: Optional[dict] # {similar_skus, bom_cost, supplier_capacity}

    # 多模态生成结果 (Media Agent 写入)
    generated_designs: Optional[list] # [{image_url, tech_pack, elements}]

    # 成本校验结果
    cost_validation: Optional[dict] # {total_cost, is_within_budget, breakdown}

    # 人机协作标记
    pending_review: bool # 是否需要人工审核
    review_notes: Optional[str] # 人工审核备注

    # 最终输出
    final_style_spec: Optional[dict] # 最终款式规格
```

```

final_plm_id: Optional[str]          # PLM 系统中的款式 ID

# 每个 Agent 节点都可以读写 State
async def trend_agent(state: PLMAgentState) -> PLMAgentState:
    """趋势分析节点 - 写入 trend_data"""
    trends = await mcp.call_tool("trend", "search_social_trends", {
        "keyword": state["design_brief"]
    })
    return {
        "trend_data": trends,
        "current_step": "plm", # 标记进入下一阶段
        "messages": [AIMessage(content=f"趋势分析完成, 获取 {len(trends)} 条数据")]
    }

```

L2: 会话记忆 (Session Memory)

跨多轮对话保持上下文，设计师可在会话内反复调整需求。使用 Mem0 实现智能记忆管理。

```

# session_memory.py - 基于 Mem0 的会话记忆
from mem0 import Memory
import uuid

class SessionMemory:
    """管理单次设计会话 (24h) 内的上下文"""

    def __init__(self):
        self.mem0 = Memory(config={
            "vector_store": {
                "provider": "qdrant",
                "config": {"host": "localhost", "port": 6333}
            },
            "llm": {
                "provider": "openai",
                "config": {"model": "gpt-4"}
            }
        })
        self.sessions: dict[str, str] = {} # session_id -> user_id

    async def create_session(self, designer_id: str) -> str:
        """创建新设计会话"""
        session_id = f"session_{uuid.uuid4().hex[:12]}"
        self.sessions[session_id] = designer_id

        await self.mem0.add(
            messages=[{
                "role": "system",
                "content": f"新设计会话开始。设计师: {designer_id}, "
                           f"时间: {datetime.now().isoformat()}"
            }],
            user_id=designer_id,
            metadata={"session_id": session_id, "type": "session_start"}
        )
        return session_id

    async def remember_design_decision(self, session_id: str, decision: dict):
        """记录关键设计决策"""
        await self.mem0.add(
            messages=[{
                "role": "user",
                "content": f"设计决策: {json.dumps(decision, ensure_ascii=False)}"
            }],
            user_id=self.sessions[session_id],
            metadata={
                "session_id": session_id,
                "type": "design_decision",
                "timestamp": datetime.now().isoformat()
            }
        )

    async def recall_session_context(self, session_id: str, query: str) -> list:
        """召回相关会话上下文"""
        memories = await self.mem0.search(
            query=query,
            user_id=self.sessions[session_id],
            limit=10
        )
        return [
            m["memory"] for m in memories
            if m.get("metadata", {}).get("session_id") == session_id
        ]

    async def summarize_session(self, session_id: str) -> str:
        """会话结束时生成摘要, 写入 L3 长期记忆"""
        session_memories = await self.mem0.get_all(
            user_id=self.sessions[session_id],
            filters={"session_id": session_id}
        )
        # 用 LLM 生成摘要...
        summary = await llm.ainvoke(
            f"总结以下设计会话的关键决策和产出:\n{session_memories}"
        )
        return summary

```

L3: 长期事件记忆 (Long-term Event Memory)

使用 Zep 图记忆引擎，将每次选品、打版、复盘的关键事件持久化，支持跨会话的知识复用。

```
# long_term_memory.py - 基于 Zep + 知识图谱的长期记忆
from zep_python import ZepClient
from zep_python.memory import Memory, Message, Session
import networkx as nx

class LongTermMemory:
    """企业级长期记忆 -- 永久沉淀设计知识与经验"""

    def __init__(self):
        self.zep = ZepClient(
            api_key=os.getenv("ZEP_API_KEY"),
            base_url=os.getenv("ZEP_BASE_URL", "http://localhost:8000")
        )
        self.knowledge_graph = nx.DiGraph() # 业务知识图谱
        self.qdrant = QdrantClient(host="localhost", port=6333)

    async def archive_design_session(self, user_id: str, session_summary: dict):
        """将完成的会话归档到 Zep 长期记忆"""
        session = Session(
            session_id=f"archived_{user_id}_{datetime.now().strftime('%Y%m%d%H%M')}",
            user_id=user_id,
            metadata={
                "design_id": session_summary["design_id"],
                "category": session_summary["category"],
                "season": session_summary["season"],
                "final_cost": session_summary["cost"],
                "style_tags": session_summary["tags"]
            }
        )

        # 写入 Zep
        await self.zep.memory.add_memory(session.session_id, Memory(
            messages=[
                Message(role="assistant", content=session_summary["summary"]),
                Message(role="system", content=f"设计规格: {session_summary['spec']}")
            ],
            summary=session_summary["summary"],
            facts=[
                f"面料选择: {session_summary['fabric']}",
                f"成本: ¥{session_summary['cost']}",
                f"版型: {session_summary['pattern']}"
            ]
        ))

        # 更新知识图谱
        self._update_knowledge_graph(session_summary)

        # 向量化存储 (供语义检索)
        await self._vectorize_and_store(session_summary)

    async def recall_similar_designs(self, query: str, top_k: int = 5):
        """召回历史相似设计案例"""
        # 向量检索
        vectors = await self.qdrant.search(
            collection_name="design_history",
            query_vector=await self._embed(query),
            limit=top_k
        )

        # 图谱增强检索 (GraphRAG)
        related_entities = self._traverse_knowledge_graph(query, depth=2)

        return {
            "similar_designs": vectors,
            "related_knowledge": related_entities
        }

    async def accumulate_design_insight(self, insight: dict):
        """沉淀设计洞察--如 '牛仔品类夏季版型偏宽松, 冬季偏修身'"""
        await self.zep.memory.add_memory("global_insights", Memory(
            messages=[Message(
                role="system",
                content=f"设计洞察 [{insight['category']}] : {insight['insight']}"
            )],
            facts=[insight["insight"]]
        ))

    def _update_knowledge_graph(self, session: dict):
        """构建设计知识图谱"""
        design_id = session["design_id"]
        # 节点: 款式
        self.knowledge_graph.add_node(design_id, type="design", **session)
        # 边: 面料关系
        for fabric in session.get("fabrics", []):
            self.knowledge_graph.add_edge(design_id, fabric, relation="uses_fabric")
        # 边: 趋势关系
        for trend in session.get("trends", []):
            self.knowledge_graph.add_edge(design_id, trend, relation="inspired_by")

    def _traverse_knowledge_graph(self, entity: str, depth: int = 2) -> list:
        """在图谱中遍历相关知识"""
        if entity not in self.knowledge_graph:
            return []
        neighbors = list(nx.dfs_successors(
```

```
        self.knowledge_graph, entity, depth_limit=depth
    ))
    return [{"entity": n, "data": self.knowledge_graph.nodes[n]} for n in neighbors]
```

三层 Memory 协作流程

- 1
- 任务启动：创建 L1 工作记忆 (LangGraph State) + 关联 L2 会话记忆 (Mem0 session_id)
- 2
- 执行中：Agent 每步读写 L1；关键决策写入 L2；查询 L3 获取历史经验
- 3
- 人机协作：设计师反馈写入 L2，Agent 从 L2 召回之前的调整历史
- 4
- 会话结束：L2 摘要写入 L3；L2 24h 后自动清理；L1 销毁
- 5
- 跨会话复用：新会话自动从 L3 检索相似案例，实现「经验传承」

Memory 技术选型对比

方案	L1 工作记忆	L2 会话记忆	L3 长期记忆
首选	LangGraph State	Mem0 + Redis	Zep + Qdrant + PG
备选	Redis String	LangChain Memory	Mem0 + Pinecone
淘汰	Context Window 全塞	纯 Redis List	纯关系型 SQL 查询
关键能力	结构化读写	语义检索 + 自动摘要	图谱推理 + 向量检索

核心价值：三层 Memory 让 Agent 从「每次都重新摸索」变成「越用越聪明」。长期记忆中的设计洞察会在每次新任务中被自动召回，形成 经验 to 知识 to 决策优化 的正向飞轮。

全流程泳道图：4 泳道并行协作

泳道定义

以四个核心角色为泳道，展示从 需求输入 to AI 执行 to 人机协作 to PLM 落库 的完整时序流程。

阶段	设计师 / 选品专员	Multi-Agent 编排引擎	MCP 工具服务层	PLM 系统
需求输入与解析 Phase 1	上传社媒穿搭图 + 输入设计需求 "生成3款2026夏季 辣妹牛仔短裤， 成本≤35元"	总管Agent解析需求 拆解为4个子任务		
		创建 LangGraph State 初始化 L1 工作记忆		
		从 L2 会话记忆 召回当前会话上下文	Mem0 API 语义检索	
		从 L3 长期记忆 召回历史相似设计案例	Zep + Qdrant 向量检索+图谱遍历	
		分发子任务给 4个专业Agent [并行执行]		

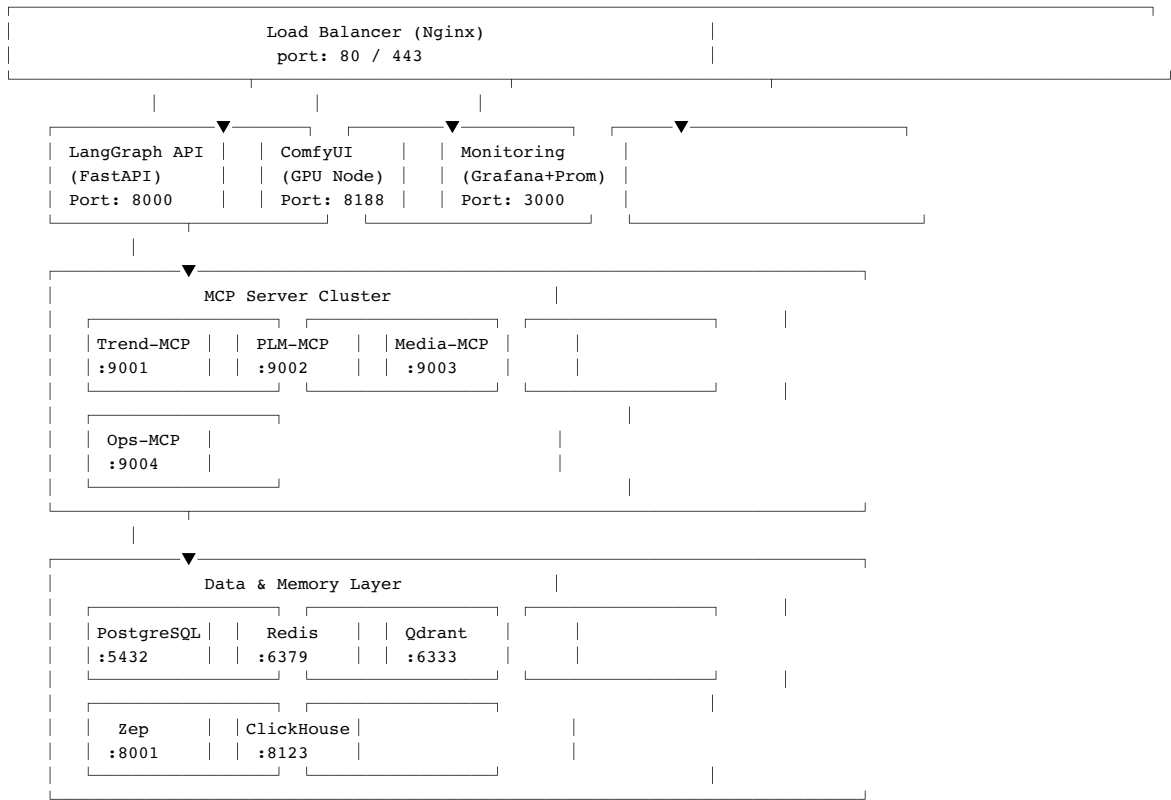
Phase 2 并行分析 与生成		Trend Agent 调用 Trend-MCP	search_social_trends fetch_trending_hashtags analyze_sentiment	
		PLM Data Agent 调用 PLM-MCP	query_sku_history get_bom_cost get_supplier_capacity	返回历史SKU数据 返回BOM成本 返回工厂产能
		Media Agent 调用 Media-MCP	generate_design_image extract_design_elements	
		Ops Agent 调用 Ops-MCP	get_sales_performance analyze_return_reasons	
		汇聚4个Agent结果 写入 L1 工作记忆		
		成本校验Agent 核算总成本是否≤35元 生成成本分解表		
		成本通过? 是 to 继续 否 to 标记需调整面料		
		生成完整款式方案 效果图 + 版型 + 面料 + BOM + 成本估算	返回生成图片 返回 tech pack	
Phase 3 人机协作 与审核	审核3款生成方案 查看效果图/版型/成本	推送审核通知 展示方案摘要		
	修改意见: "第2款腰头太宽, 改窄2cm; 第3款换水洗工艺"	解析修改意见 调用 Media Agent 重新生成 写入 L2 会话记忆	generate_design_image (带修改参数)	
	确认最终款式	生成 PLM 写入数据 款式编码/规格/图片/BOM		
			PLM-MCP.write_style_spec	款式落库 生成 PLM ID 关联 BOM/物料/图片
Phase 4 闭环复盘 与沉淀		启动复盘任务 (定时) 上线30天后自动触发		返回该款销售数据 退货率/评分/库存
	查看复盘报告 了解哪些设计决策有效	生成复盘报告 对比预测 vs 实际 提取关键洞察	get_sales_performance analyze_return_reasons	
		沉淀设计洞察到 L3 更新知识图谱	Zep add_memory 更新知识图谱	
		下次相似需求 自动召回本次经验	Qdrant 向量检索 返回相似案例	

关键时序指标

阶段	耗时	关键依赖	并行机会
Phase 1: 需求解析	< 30s	总管 Agent LLM 推理	L2/L3 记忆检索可并行
Phase 2: 并行分析	< 3min	Media Agent 图片生成（最慢）	4 Agent 完全并行
Phase 3: 人机协作	5-30min	设计师审核时间	审核期间 Agent 可处理其他任务
Phase 4: 闭环复盘	上线后30天自动触发	销售数据积累	独立后台任务，不阻塞
端到端（无人工等待）	< 5min	从需求输入到 PLM 款式落库	

部署架构：从本地开发到生产集群

整体部署架构图



Docker Compose – 一键启动开发环境

```
# docker-compose.yml - 完整 PLM Agent 技术栈
title: "SHEIN PLM AI Agent Stack"

services:
  # ===== 核心 API =====
  langgraph-api:
    build:
      context: ./langgraph
      dockerfile: Dockerfile
    ports:
      - "8000:8000"
    environment:
      - OPENAI_API_KEY=${OPENAI_API_KEY}
      - ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY}
      - MCP_TREND_URL=http://trend-mcp:9001
      - MCP_PLM_URL=http://plm-mcp:9002
      - MCP_MEDIA_URL=http://media-mcp:9003
      - MCP_OPS_URL=http://ops-mcp:9004
      - REDIS_URL=redis://redis:6379
      - POSTGRES_URL=postgres://plm:plm123@postgres:5432/plm_agent
      - QDRANT_URL=http://qdrant:6333
      - ZEP_API_URL=http://zep:8001
    depends_on:
      redis:
        condition: service_healthy
      postgres:
        condition: service_healthy
      qdrant:
        condition: service_healthy
    volumes:
      - ./langgraph:/app
    command: uvicorn api:app --host 0.0.0.0 --port 8000 --reload
    networks:
```

```

- plm-network

# ===== MCP Server 集群 =====
trend-mcp:
  build:
    context: ./mcp_servers/trend
    dockerfile: Dockerfile
  ports:
    - "9001:9001"
  environment:
    - INSTAGRAM_TOKEN=${INSTAGRAM_TOKEN}
    - PINTEREST_TOKEN=${PINTEREST_TOKEN}
    - TIKTOK_TOKEN=${TIKTOK_TOKEN}
  volumes:
    - ./mcp_servers/trend:/app
  command: python trend_mcp_server.py --transport sse --port 9001
  networks:
    - plm-network

plm-mcp:
  build:
    context: ./mcp_servers/plm
    dockerfile: Dockerfile
  ports:
    - "9002:9002"
  environment:
    - PLM_DB_URL=postgres://plm:plm123@postgres:5432/plm
    - QDRANT_URL=http://qdrant:6333
    - ERP_API_KEY=${ERP_API_KEY}
  depends_on:
    postgres:
      condition: service_healthy
  volumes:
    - ./mcp_servers/plm:/app
  command: python plm_mcp_server.py --transport sse --port 9002
  networks:
    - plm-network

media-mcp:
  build:
    context: ./mcp_servers/media
    dockerfile: Dockerfile
  ports:
    - "9003:9003"
  environment:
    - COMFYUI_URL=http://comfyui:8188
    - HF_TOKEN=${HF_TOKEN}
  depends_on:
    - comfyui
  volumes:
    - ./mcp_servers/media:/app
    - media_output:/data/output
  command: python media_mcp_server.py --transport sse --port 9003
  networks:
    - plm-network
  deploy:
    resources:
      reservations:
        devices:
          - driver: nvidia
            count: 1
            capabilities: [gpu]

ops-mcp:
  build:
    context: ./mcp_servers/ops
    dockerfile: Dockerfile
  ports:
    - "9004:9004"
  environment:
    - CLICKHOUSE_URL=http://clickhouse:8123
    - BI_API_KEY=${BI_API_KEY}
  depends_on:
    clickhouse:
      condition: service_healthy
  volumes:
    - ./mcp_servers/ops:/app
  command: python ops_mcp_server.py --transport sse --port 9004
  networks:
    - plm-network

# ===== AI 模型服务 =====
comfyui:
  image: ghcr.io/ai-dock/comfyui:latest
  ports:
    - "8188:8188"
  environment:
    - NVIDIA_VISIBLE_DEVICES=all
  volumes:
    - ./comfyui/models:/opt/ComfyUI/models
    - ./comfyui/workflows:/opt/ComfyUI/user/default/workflows
    - media_output:/opt/ComfyUI/output
  deploy:
    resources:
      reservations:
        devices:
          - driver: nvidia
            count: 1
            capabilities: [gpu]
  networks:

```

```

- plm-network

# ===== 数据层 =====
postgres:
  image: pgvector/pgvector:pg16
  ports:
    - "5432:5432"
  environment:
    POSTGRES_DB: plm_agent
    POSTGRES_USER: plm
    POSTGRES_PASSWORD: plm123
  volumes:
    - postgres_data:/var/lib/postgresql/data
    - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U plm -d plm_agent"]
    interval: 5s
    timeout: 5s
    retries: 5
  networks:
    - plm-network

redis:
  image: redis/redis-stack-server:latest
  ports:
    - "6379:6379"
  volumes:
    - redis_data:/data
  healthcheck:
    test: ["CMD", "redis-cli", "ping"]
    interval: 5s
    timeout: 5s
    retries: 5
  networks:
    - plm-network

qdrant:
  image: qdrant/qdrant:latest
  ports:
    - "6333:6333"
    - "6334:6334"
  volumes:
    - qdrant_data:/qdrant/storage
  networks:
    - plm-network

zep:
  image: zepai/zep:latest
  ports:
    - "8001:8001"
  environment:
    - ZEP_CONFIG_PATH=/app/config.yaml
  volumes:
    - ./zep/config.yaml:/app/config.yaml
    - zep_data:/app/data
  networks:
    - plm-network

clickhouse:
  image: clickhouse/clickhouse-server:latest
  ports:
    - "8123:8123"
    - "9000:9000"
  environment:
    CLICKHOUSE_DB: plm_analytics
    CLICKHOUSE_USER: plm
    CLICKHOUSE_PASSWORD: plm123
  volumes:
    - clickhouse_data:/var/lib/clickhouse
  healthcheck:
    test: ["CMD-SHELL", "clickhouse-client --query 'SELECT 1'"]
    interval: 5s
    timeout: 5s
    retries: 5
  networks:
    - plm-network

# ===== 监控栈 =====
prometheus:
  image: prom/prometheus:latest
  ports:
    - "9090:9090"
  volumes:
    - ./monitoring/prometheus.yml:/etc/prometheus/prometheus.yml
    - prometheus_data:/prometheus
  command:
    - '--config.file=/etc/prometheus/prometheus.yml'
    - '--storage.tsdb.path=/prometheus'
  networks:
    - plm-network

grafana:
  image: grafana/grafana:latest
  ports:
    - "3000:3000"
  environment:
    - GF_SECURITY_ADMIN_PASSWORD=${GRAFANA_PASSWORD:-admin}
  volumes:
    - ./monitoring/grafana/dashboards:/etc/grafana/provisioning/dashboards
    - grafana_data:/var/lib/grafana

```

```

    depends_on:
      - prometheus
    networks:
      - plm-network

# ===== 消息队列 =====
nats:
  image: nats:latest
  ports:
    - "4222:4222"
    - "8222:8222"
  command: "-js"
  networks:
    - plm-network

networks:
  plm-network:
    driver: bridge

volumes:
  postgres_data:
  redis_data:
  qdrant_data:
  zep_data:
  clickhouse_data:
  prometheus_data:
  grafana_data:
  media_output:

```

Kubernetes 生产部署

```

# k8s/langgraph-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: langgraph-api
  namespace: plm-agent
spec:
  replicas: 3
  selector:
    matchLabels:
      app: langgraph-api
  template:
    metadata:
      labels:
        app: langgraph-api
    spec:
      containers:
        - name: langgraph
          image: registry.shein.com/plm-agent/langgraph:latest
          ports:
            - containerPort: 8000
          env:
            - name: OPENAI_API_KEY
              valueFrom:
                secretKeyRef:
                  name: ai-secrets
                  key: openai-key
            - name: REDIS_URL
              value: "redis://redis-service:6379"
            - name: MCP_TREND_URL
              value: "http://trend-mcp-service:9001"
            - name: MCP_PLM_URL
              value: "http://plm-mcp-service:9002"
            - name: MCP_MEDIA_URL
              value: "http://media-mcp-service:9003"
            - name: MCP_OPS_URL
              value: "http://ops-mcp-service:9004"
          resources:
            requests:
              memory: "512Mi"
              cpu: "500m"
            limits:
              memory: "2Gi"
              cpu: "2000m"
          livenessProbe:
            httpGet:
              path: /health
              port: 8000
            initialDelaySeconds: 30
            periodSeconds: 10
          readinessProbe:
            httpGet:
              path: /ready
              port: 8000
            initialDelaySeconds: 5
            periodSeconds: 5
      ---
    apiVersion: v1
    kind: Service
    metadata:
      name: langgraph-service
      namespace: plm-agent
    spec:
      selector:
        app: langgraph-api
      ports:
        - port: 8000

```

```
targetPort: 8000
type: ClusterIP

---
# HPA 自动扩缩容
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: langgraph-hpa
  namespace: plm-agent
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: langgraph-api
  minReplicas: 3
  maxReplicas: 20
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
  - type: Resource
    resource:
      name: memory
      target:
        type: Utilization
        averageUtilization: 80

---
# GPU 节点专用部署 (ComfyUI)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: comfyui-gpu
  namespace: plm-agent
spec:
  replicas: 2
  selector:
    matchLabels:
      app: comfyui
  template:
    spec:
      nodeSelector:
        accelerator: nvidia-a100
      containers:
      - name: comfyui
        image: ghcr.io/ai-dock/comfyui:latest
        ports:
        - containerPort: 8188
        resources:
          limits:
            nvidia.com/gpu: 1
          volumeMounts:
          - name: models
            mountPath: /opt/ComfyUI/models
        volumes:
        - name: models
          persistentVolumeClaim:
            claimName: comfyui-models-pvc
```

监控与可观测性

层级	工具	监控内容	告警规则
LLM 链路追踪	LangSmith / OpenTelemetry	每次 Agent 调用的 Token 用量、延迟、工具调用链、错误率	Token 超限、LLM 返回错误率 > 5%
API 层	Prometheus + Grafana	QPS、P50/P95/P99 延迟、错误率、并发数	P95 > 10s、错误率 > 1%、Pod 重启 > 3次/h
MCP Server Health Check + Prometheus		各 MCP Server 可用性、单工具调用延迟、连接数	Server Down > 30s、工具超时率 > 10%
数据层	PostgreSQL Exporter / Redis Exporter	连接池、慢查询、缓存命中率、磁盘用量	连接池耗尽、慢查询 > 1s、磁盘 > 80%
GPU 节点	NVIDIA DCGM + Prometheus	GPU 利用率、显存、温度、功耗	GPU 利用率 > 95% 持续 5min、显存不足
业务指标	ClickHouse + Grafana	每日选品数、PLM 写入成功率、人工审核通过率、平均选品耗时	选品耗时 > 10min、PLM 写入失败 > 10次/h

CI/CD 流水线

```
# .github/workflows/deploy.yml
name: Deploy PLM Agent

on:
  push:
    branches: [main]
    paths:
      - 'langgraph/**'
      - 'mcp_servers/**'
```

```
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.11' }
      - run: pip install -r requirements-dev.txt
      - run: pytest tests/ -v --cov
      - run: ruff check .

  build-and-push:
    needs: test
    runs-on: ubuntu-latest
    strategy:
      matrix:
        service: [langgraph, trend-mcp, plm-mcp, media-mcp, ops-mcp]
    steps:
      - uses: actions/checkout@v4
      - name: Build & Push
        run: |
          docker build -t registry.shein.com/plm-agent/${{ matrix.service }}:${{ github.sha }} \
            -f ${{{ matrix.service }}/Dockerfile} .
          docker push registry.shein.com/plm-agent/${{ matrix.service }}:${{ github.sha }}

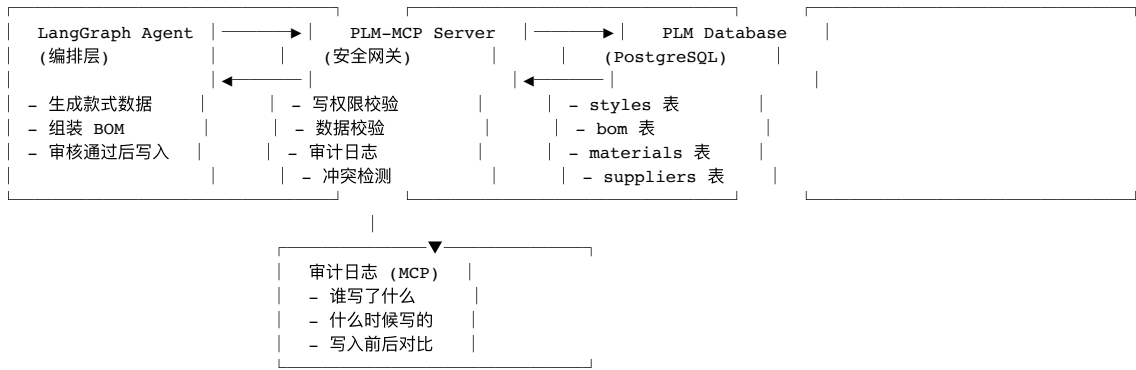
  deploy:
    needs: build-and-push
    runs-on: ubuntu-latest
    steps:
      - uses: azure/k8s-deploy@v4
        with:
          manifests: k8s/*.yaml
          images: |
            registry.shein.com/plm-agent/langgraph:${{ github.sha }}
            registry.shein.com/plm-agent/trend-mcp:${{ github.sha }}
```

教程：PLM 同步 MCP – 写回与双向同步

核心挑战

SHEIN PLM 系统是企业核心，AI Agent 需要安全、可靠、可追溯地写入数据。本教程详细讲解如何实现 **PLM-MCP** 的写回能力，包括款式创建、BOM 同步、版型规格、以及数据冲突处理。

PLM 同步架构



Step 1: PLM 数据模型定义

```
# plm_models.py - PLM 核心数据模型
from sqlalchemy import (
    Column, String, Integer, Float, DateTime, Text,
    JSON, ForeignKey, Enum as SAEnum, Boolean
)
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.dialects.postgresql import UUID, ARRAY
from datetime import datetime
import enum
import uuid

Base = declarative_base()
schema = "plm"

class StyleStatus(str, enum.Enum):
    DRAFT = "draft" # AI 生成草稿
    PENDING_REVIEW = "pending" # 待人工审核
    APPROVED = "approved" # 审核通过
    REJECTED = "rejected" # 已驳回
    IN_PRODUCTION = "production" # 已投产
    ARCHIVED = "archived" # 已归档

class Style(Base):
```

```

"""款式主表"""
__tablename__ = "styles"
__table_args__ = {"schema": schema}

id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
style_code = Column(String(50), unique=True, nullable=False, index=True)
design_name = Column(String(200), nullable=False)
category = Column(String(100), nullable=False, index=True)
sub_category = Column(String(100))
season = Column(String(20), nullable=False, index=True)

# 设计信息
design_images = Column(JSON, default=list) # ["url1", "url2"]
description = Column(Text)
inspiration_refs = Column(JSON, default=list)
trend_keywords = Column(ARRAY(String))

# 版型规格
size_spec = Column(JSON, default=dict) # {"XS": {"waist": 64, ...}, ...}
pattern_type = Column(String(100))
fit_type = Column(String(50)) # slim, regular, loose, oversized

# 成本
target_cost = Column(Float)
estimated_cost = Column(Float)
actual_cost = Column(Float)
currency = Column(String(3), default="CNY")

# 状态与审计
status = Column(SAEnum(StyleStatus), default=StyleStatus.DRAFT, index=True)
created_by = Column(String(100), default="ai-agent")
created_at = Column(DateTime, default=datetime.utcnow)
updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)
reviewed_by = Column(String(100))
reviewed_at = Column(DateTime)

# AI 元数据
ai_generation_meta = Column(JSON, default=dict) # Agent版本、prompt、模型等
ai_confidence_score = Column(Float) # AI 对款式成功率的预估

class BOMItem(Base):
    """物料清单明细"""
    __tablename__ = "bom_items"
    __table_args__ = {"schema": schema}

    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    style_id = Column(UUID(as_uuid=True), ForeignKey(f"{schema}.styles.id"), nullable=False, index=True)

    material_code = Column(String(50), nullable=False)
    material_name = Column(String(200), nullable=False)
    material_type = Column(String(50)) # fabric, accessory, trim, packaging
    color = Column(String(50))

    quantity_per_unit = Column(Float)
    unit = Column(String(20)) # m, kg, pcs
    unit_cost = Column(Float)
    total_cost = Column(Float)

    supplier_id = Column(UUID(as_uuid=True))
    supplier_name = Column(String(200))
    lead_time_days = Column(Integer)

    is_ai_suggested = Column(Boolean, default=True) # AI 推荐的面料
    alternative_material_code = Column(String(50)) # 备选物料

class WriteAuditLog(Base):
    """写入审计日志"""
    __tablename__ = "write_audit_logs"
    __table_args__ = {"schema": schema}

    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    timestamp = Column(DateTime, default=datetime.utcnow, index=True)

    operation = Column(String(50)) # INSERT, UPDATE, DELETE
    table_name = Column(String(100))
    record_id = Column(String(100))

    before_data = Column(JSON) # 变更前
    after_data = Column(JSON) # 变更后

    performed_by = Column(String(100)) # ai-agent / designer_id
    agent_version = Column(String(50))
    session_id = Column(String(100))

    ip_address = Column(String(50))
    user_agent = Column(String(200))

```

Step 2: PLM-MCP 写回工具实现

```

# plm_write_tools.py - PLM 写回核心逻辑
from sqlalchemy.ext.asyncio import AsyncSession, create_async_engine
from sqlalchemy import select, update, and_
from sqlalchemy.dialects.postgresql import insert
from datetime import datetime
import json

```

```

class PLMWriteService:
    """
    PLM 写回服务 -- 安全、可靠、可追溯

    设计原则：
    1. 所有写操作必须经过数据校验
    2. 所有写操作必须记录审计日志
    3. AI 写入默认为 draft 状态，需人工审核
    4. 支持乐观锁冲突检测
    """

    def __init__(self, db_url: str):
        self.engine = create_async_engine(db_url)

    async def create_style(self, style_data: dict, session_id: str) -> dict:
        """
        创建新款式到 PLM

        Args:
            style_data: 款式完整数据
            session_id: Agent 会话 ID (用于审计追溯)

        Returns:
            {"status": "success", "style_id": "...", "style_code": "..."}
        """
        # 1. 数据校验
        validation_errors = self._validate_style_data(style_data)
        if validation_errors:
            return {"status": "validation_failed", "errors": validation_errors}

        # 2. 生成款式编码
        style_code = self._generate_style_code(style_data)

        async with AsyncSession(self.engine) as session:
            async with session.begin():
                # 3. 检查重复 (款名+品类+季节)
                existing = await session.execute(
                    select(Style).where(
                        and_(
                            Style.design_name == style_data["design_name"],
                            Style.category == style_data["category"],
                            Style.season == style_data["season"]
                        )
                    )
                )
                if existing.scalar_one_or_none():
                    return {
                        "status": "duplicate",
                        "message": f"款式 '{style_data['design_name']}' 已存在"
                    }

                # 4. 写入款式主表
                style = Style(
                    style_code=style_code,
                    design_name=style_data["design_name"],
                    category=style_data["category"],
                    sub_category=style_data.get("sub_category"),
                    season=style_data["season"],
                    design_images=style_data.get("design_images", []),
                    description=style_data.get("description", ""),
                    inspiration_refs=style_data.get("inspiration_refs", []),
                    trend_keywords=style_data.get("trend_keywords", []),
                    size_spec=style_data.get("size_spec", {}),
                    pattern_type=style_data.get("pattern_type"),
                    fit_type=style_data.get("fit_type"),
                    target_cost=style_data.get("target_cost"),
                    estimated_cost=style_data.get("estimated_cost"),
                    status=StyleStatus.DRAFT,
                    created_by=f"ai-agent:{session_id}",
                    ai_generation_meta=style_data.get("ai_meta", {}),
                    ai_confidence_score=style_data.get("confidence_score")
                )
                session.add(style)
                await session.flush() # 获取 style.id

                # 5. 写入 BOM
                for bom_item in style_data.get("bom", []):
                    session.add(BOMItem(
                        style_id=style.id,
                        material_code=bom_item["material_code"],
                        material_name=bom_item["material_name"],
                        material_type=bom_item.get("material_type"),
                        color=bom_item.get("color"),
                        quantity_per_unit=bom_item.get("quantity_per_unit"),
                        unit=bom_item.get("unit", "pcs"),
                        unit_cost=bom_item.get("unit_cost"),
                        total_cost=bom_item.get("total_cost"),
                        supplier_id=bom_item.get("supplier_id"),
                        supplier_name=bom_item.get("supplier_name"),
                        lead_time_days=bom_item.get("lead_time_days"),
                        is_ai_suggested=True
                    ))

                # 6. 写入审计日志
                session.add(WriteAuditLog(
                    operation="INSERT",
                    table_name="styles",

```

```

        record_id=str(style.id),
        after_data=self._serialize_style(style),
        performed_by=f"ai-agent:{session_id}",
        session_id=session_id
    ))

    await session.commit()

    return {
        "status": "success",
        "style_id": str(style.id),
        "style_code": style_code,
        "message": f"款式 '{style_data['design_name']}' 已创建 (状态: draft)"
    }

async def update_style(self, style_id: str, updates: dict,
                      performed_by: str, session_id: str) -> dict:
    """
    更新已有款式 (仅 draft 或 pending 状态可修改)
    """
    async with AsyncSession(self.engine) as session:
        async with session.begin():
            # 1. 读取当前数据 (用于审计对比)
            result = await session.execute(
                select(Style).where(Style.id == style_id)
            )
            style = result.scalar_one_or_none()

            if not style:
                return {"status": "not_found", "message": f"款式 {style_id} 不存在"}

            before_data = self._serialize_style(style)

            # 2. 状态检查: 只有 draft/pending 可修改
            if style.status not in (StyleStatus.DRAFT, StyleStatus.PENDING_REVIEW):
                return {
                    "status": "forbidden",
                    "message": f"款式状态为 {style.status.value}, 不允许修改"
                }

            # 3. 更新字段 (仅允许更新特定字段)
            allowed_fields = {
                "design_name", "description", "design_images", "size_spec",
                "pattern_type", "fit_type", "estimated_cost", "trend_keywords"
            }
            for key, value in updates.items():
                if key in allowed_fields:
                    setattr(style, key, value)

            style.updated_at = datetime.utcnow()

            # 4. 审计日志
            after_data = self._serialize_style(style)
            session.add(WriteAuditLog(
                operation="UPDATE",
                table_name="styles",
                record_id=style_id,
                before_data=before_data,
                after_data=after_data,
                performed_by=performed_by,
                session_id=session_id
            ))

        await session.commit()

    return {
        "status": "success",
        "style_id": style_id,
        "updated_fields": list(set(updates.keys()) & allowed_fields)
    }

async def approve_style(self, style_id: str, reviewer_id: str) -> dict:
    """人工审核通过 to 状态变更为 approved"""
    async with AsyncSession(self.engine) as session:
        async with session.begin():
            result = await session.execute(
                select(Style).where(Style.id == style_id)
            )
            style = result.scalar_one_or_none()

            if not style:
                return {"status": "not_found"}

            style.status = StyleStatus.APPROVED
            style.reviewed_by = reviewer_id
            style.reviewed_at = datetime.utcnow()

            session.add(WriteAuditLog(
                operation="STATUS_CHANGE",
                table_name="styles",
                record_id=style_id,
                before_data={"status": "pending"},
                after_data={"status": "approved"},
                performed_by=reviewer_id
            ))

        await session.commit()

    return {"status": "success", "style_id": style_id, "new_status": "approved"}

```

```

async def sync_bom_from_ai(self, style_id: str, bom_data: list[dict],
                           session_id: str) -> dict:
    """
    AI 同步 BOM 数据到 PLM
    - 新物料: INSERT
    - 已有物料: UPDATE
    - 删除物料: 标记 is_active=False
    """
    async with AsyncSession(self.engine) as session:
        async with session.begin():
            # 获取现有 BOM
            existing = await session.execute(
                select(BOMItem).where(BOMItem.style_id == style_id)
            )
            existing_items = {item.material_code: item for item in existing.scalars()}

            new_codes = set()
            results = {"inserted": 0, "updated": 0, "deleted": 0}

            for bom_entry in bom_data:
                code = bom_entry["material_code"]
                new_codes.add(code)

                if code in existing_items:
                    # 更新
                    item = existing_items[code]
                    for key in ["unit_cost", "total_cost", "quantity_per_unit", "lead_time_days"]:
                        if key in bom_entry:
                            setattr(item, key, bom_entry[key])
                    results["updated"] += 1
                else:
                    # 新增
                    session.add(BOMItem(
                        style_id=style_id,
                        material_code=code,
                        material_name=bom_entry["material_name"],
                        material_type=bom_entry.get("material_type"),
                        quantity_per_unit=bom_entry.get("quantity_per_unit"),
                        unit=bom_entry.get("unit", "pcs"),
                        unit_cost=bom_entry.get("unit_cost"),
                        total_cost=bom_entry.get("total_cost"),
                        is_ai_suggested=True
                    ))
                    results["inserted"] += 1

            # 软删除: AI 不再建议的物料
            for code, item in existing_items.items():
                if code not in new_codes and item.is_ai_suggested:
                    item.is_active = False
                    results["deleted"] += 1

            # 审计
            session.add(WriteAuditLog(
                operation="BOM_SYNC",
                table_name="bom_items",
                record_id=style_id,
                after_data={"summary": results},
                performed_by=f"ai-agent:{session_id}",
                session_id=session_id
            ))

        await session.commit()

    return {"status": "success", **results}

def _validate_style_data(self, data: dict) -> list[str]:
    """数据校验规则"""
    errors = []
    required = ["design_name", "category", "season"]
    for field in required:
        if not data.get(field):
            errors.append(f"缺少必填字段: {field}")

    if data.get("estimated_cost") and data["estimated_cost"] < 0:
        errors.append("预估成本不能为负数")

    if data.get("target_cost") and data.get("estimated_cost"):
        if data["estimated_cost"] > data["target_cost"] * 1.3:
            errors.append(
                f"预估成本 ¥{data['estimated_cost']} "
                f"超出目标成本 ¥{data['target_cost']} 的 30%"
            )

    return errors

def _generate_style_code(self, data: dict) -> str:
    """生成款式编码: CAT-SEASON-NNNNN"""
    cat_abbr = data["category"][:4].upper()
    season_abbr = data["season"].upper()
    seq = f"{datetime.utcnow().strftime('%m%d%H%M')}"
    return f"{cat_abbr}-{season_abbr}-{seq}"

def _serialize_style(self, style: Style) -> dict:
    """序列化款式对象 (用于审计日志)"""
    return {
        "style_code": style.style_code,
    }

```

```

        "design_name": style.design_name,
        "category": style.category,
        "status": style.status.value,
        "estimated_cost": style.estimated_cost,
        "target_cost": style.target_cost
    }

# ===== 集成到 MCP Server =====
# plm_mcp_server.py 中添加写工具

@server.list_tools()
async def list_tools():
    return [
        # ... 之前的读工具 ...
        Tool(
            name="write_style_spec",
            description="🚧 写入操作: 创建新款式到 PLM 系统。款式以 draft 状态入库, 需人工审核后才能进入生产。",
            inputSchema={
                "type": "object",
                "properties": {
                    "style_data": {
                        "type": "object",
                        "description": "完整的款式数据",
                        "properties": {
                            "design_name": {"type": "string"},
                            "category": {"type": "string"},
                            "season": {"type": "string"},
                            "design_images": {"type": "array", "items": {"type": "string"}},
                            "description": {"type": "string"},
                            "trend_keywords": {"type": "array", "items": {"type": "string"}},
                            "size_spec": {"type": "object"},
                            "pattern_type": {"type": "string"},
                            "fit_type": {"type": "string"},
                            "target_cost": {"type": "number"},
                            "estimated_cost": {"type": "number"},
                            "bom": {
                                "type": "array",
                                "items": {
                                    "type": "object",
                                    "properties": {
                                        "material_code": {"type": "string"},
                                        "material_name": {"type": "string"},
                                        "material_type": {"type": "string"},
                                        "quantity_per_unit": {"type": "number"},
                                        "unit": {"type": "string"},
                                        "unit_cost": {"type": "number"},
                                        "total_cost": {"type": "number"}
                                    }
                                },
                                "required": ["material_code", "material_name", "unit_cost"]
                            }
                        },
                        "required": ["material_code", "material_name", "unit_cost"]
                    },
                    "confidence_score": {"type": "number"}
                },
                "required": ["design_name", "category", "season", "bom"]
            },
            "session_id": {"type": "string", "description": "Agent 会话 ID"}
        },
        {
            "required": ["style_data", "session_id"]
        }
    ],
    Tool(
        name="update_style",
        description="更新已有款式的设计数据 (仅 draft/pending 状态)",
        inputSchema={
            "type": "object",
            "properties": {
                "style_id": {"type": "string"},
                "updates": {"type": "object"},
                "session_id": {"type": "string"}
            },
            "required": ["style_id", "updates", "session_id"]
        }
    )
]

plm_write_service = PLMWriteService(os.getenv("PLM_DB_URL"))

@server.call_tool()
async def call_tool(name: str, arguments: dict) -> list[TextContent]:
    # ... 读工具处理 ...

    if name == "write_style_spec":
        result = await plm_write_service.create_style(
            arguments["style_data"],
            arguments["session_id"]
        )
        return [TextContent(type="text", text=json.dumps(result, ensure_ascii=False, indent=2))]

    if name == "update_style":
        result = await plm_write_service.update_style(
            arguments["style_id"],
            arguments["updates"],
            f'ai-agent:{arguments["session_id"]}',
            arguments["session_id"]
        )
        return [TextContent(type="text", text=json.dumps(result, ensure_ascii=False, indent=2))]

```

Step 3: 冲突检测与重试策略

```
# plm_conflict_resolver.py
class PLMConflictResolver:
    """PLM 数据冲突检测与解决"""

    async def detect_conflict(self, style_id: str, ai_version: str) -> Optional[dict]:
        """
        检测 AI 写入时的数据冲突
        场景: 设计师在 AI 生成期间手动修改了 PLM 中的数据
        """
        async with AsyncSession(self.engine) as session:
            result = await session.execute(
                select(Style).where(Style.id == style_id)
            )
            style = result.scalar_one_or_none()

            if not style:
                return None

            # 检查最后更新时间和修改者
            if style.updated_at and style.ai_generation_meta:
                last_ai_version = style.ai_generation_meta.get("agent_version")
                if last_ai_version != ai_version:
                    return {
                        "conflict": True,
                        "current_version": last_ai_version,
                        "incoming_version": ai_version,
                        "last_modified": style.updated_at.isoformat(),
                        "last_modified_by": style.created_by
                    }

            return {"conflict": False}

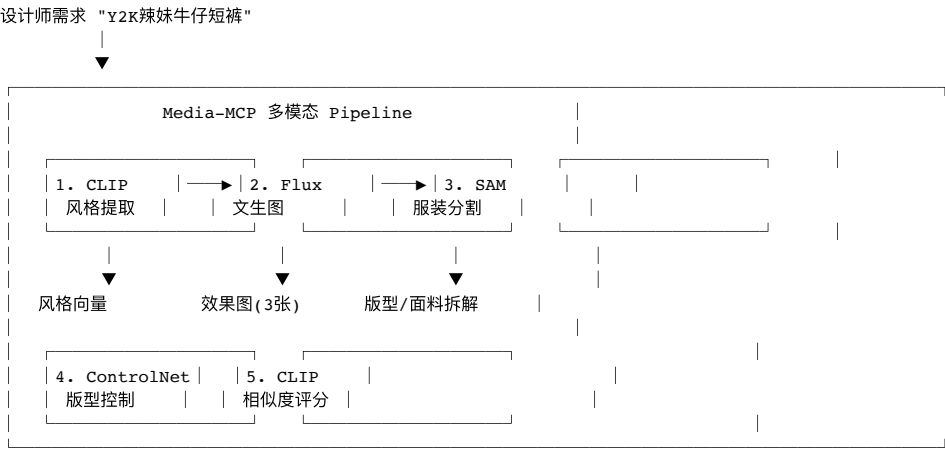
    async def resolve_with_retry(self, operation, max_retries: int = 3):
        """带重试的写操作"""
        import asyncio
        for attempt in range(max_retries):
            try:
                result = await operation()
                if result.get("status") != "conflict":
                    return result
                # 冲突: 等待后重试
                await asyncio.sleep(0.5 * (2 ** attempt))
            except Exception as e:
                if attempt == max_retries - 1:
                    raise
                await asyncio.sleep(1.0)
        return {"status": "failed", "message": "达到最大重试次数"}
```

PLM 同步监控指标

指标	目标值	告警阈值	监控方式
写操作成功率	≥ 99.9%	< 99%	Prometheus Counter
写操作 P95 延迟	< 500ms	> 2s	Prometheus Histogram
冲突率	< 1%	> 5%	冲突检测日志
数据校验失败率	< 3%	> 10%	validation_errors counter
审计日志完整率	100%	< 100%	审计日志表计数 vs 写操作数
人工审核通过率	≥ 80%	< 60%	style status 统计

教程：多模态 AI Pipeline – 文生图 + CLIP 风格提取

整体 Pipeline 架构



Step 1: 环境准备 – ComfyUI + 模型

```
# 模型下载列表
# Flux.1-dev (文生图): https://huggingface.co/black-forest-labs/FLUX.1-dev
# CLIP-ViT-L/14: https://huggingface.co/openai/clip-vit-large-patch14
# SAM (服装分割): https://huggingface.co/facebook/sam-vit-huge
# ControlNet-Canny: https://huggingface.co/lllyasviel/sd-controlnet-canny

# requirements.txt
pip install diffusers transformers accelerate
pip install open_clip_torch segment-anything
pip install comfy-cli # ComfyUI 命令行工具
```

Step 2: CLIP 风格提取 – 从参考图中提取设计元素

```
# clip_design_extractor.py
import torch
import open_clip
from PIL import Image
from typing import List, Dict
import numpy as np

class CLIPDesignExtractor:
    """
    使用 CLIP 从参考图中提取服装设计元素
    支持: 风格分类、颜色提取、廓形识别、细节检测
    """

    def __init__(self, model_name: str = "ViT-L-14", pretrained: str = "openai"):
        self.device = "cuda" if torch.cuda.is_available() else "cpu"
        self.model, _, self.preprocess = open_clip.create_model_and_transforms(
            model_name, pretrained=pretrained
        )
        self.model = self.model.to(self.device)
        self.tokenizer = open_clip.get_tokenizer(model_name)

        # 预定义的服装设计元素标签
        self.style_labels = [
            "Y2K style", "minimalist", "boho chic", "streetwear",
            "athleisure", "vintage", "cottagecore", "grunge",
            "preppy", "avant-garde", "romantic", "utility"
        ]

        self.silhouette_labels = [
            "A-line silhouette", "hourglass silhouette", "oversized silhouette",
            "fitted silhouette", "boxy silhouette", "empire waist",
            "low-rise", "high-waisted", "mid-rise"
        ]

        self.fabric_labels = [
            "denim fabric", "cotton fabric", "silk fabric",
            "leather fabric", "knit fabric", "linen fabric",
            "chiffon fabric", "velvet fabric", "distressed denim"
        ]

        self.detail_labels = [
            "ruffle detail", "lace detail", "embroidery detail",
            "frayed hem", "raw edge", "distressed detail",
            "patchwork", "asymmetric design", "cutout detail"
        ]

    def extract_style(self, image: Image.Image) -> Dict[str, float]:
        """提取风格特征向量和分类得分"""
        image_tensor = self.preprocess(image).unsqueeze(0).to(self.device)

        with torch.no_grad():
            image_features = self.model.encode_image(image_tensor)
            image_features = image_features / image_features.norm(dim=-1, keepdim=True)

        # 与风格标签计算相似度
        style_scores = {}
        for label_set, label_name in [
            (self.style_labels, "style"),
            (self.silhouette_labels, "silhouette"),
            (self.fabric_labels, "fabric"),
            (self.detail_labels, "detail")
        ]:
            text_tokens = self.tokenizer(label_set).to(self.device)
            with torch.no_grad():
                text_features = self.model.encode_text(text_tokens)
                text_features = text_features / text_features.norm(dim=-1, keepdim=True)

            similarity = (image_features @ text_features.T).squeeze(0)
            for label, score in zip(label_set, similarity.cpu().tolist()):
                style_scores[label] = round(score, 4)

        # 排序并返回 Top 3
        sorted_scores = sorted(style_scores.items(), key=lambda x: x[1], reverse=True)
        return {
            "top_styles": sorted_scores[:3],
            "image_embedding": image_features.cpu().numpy().tolist()[0],
            "all_scores": style_scores
        }

    def compare_designs(self, design_image: Image.Image,
                       reference_image: Image.Image) -> Dict:
        """对比生成图与参考图的风格相似度"""
        design_tensor = self.preprocess(design_image).unsqueeze(0).to(self.device)
        ref_tensor = self.preprocess(reference_image).unsqueeze(0).to(self.device)
```

```

with torch.no_grad():
    design_feat = self.model.encode_image(design_tensor)
    ref_feat = self.model.encode_image(ref_tensor)
    design_feat = design_feat / design_feat.norm(dim=-1, keepdim=True)
    ref_feat = ref_feat / ref_feat.norm(dim=-1, keepdim=True)

    cosine_sim = (design_feat @ ref_feat.T).item()

    return {
        "cosine_similarity": round(cosine_sim, 4),
        "interpretation": self._interpret_similarity(cosine_sim)
    }

def _interpret_similarity(self, score: float) -> str:
    if score > 0.85: return "高度相似 - 风格完美匹配参考图"
    elif score > 0.70: return "较高相似 - 核心风格元素一致"
    elif score > 0.55: return "中等相似 - 有共同风格倾向"
    else: return "低相似 - 风格差异较大, 建议调整 prompt"

def extract_design_keywords(self, image: Image.Image, top_k: int = 5) -> List[str]:
    """从图片自动提取设计关键词 (用于生成 prompt) """
    scores = self.extract_style(image)
    keywords = []
    for label, score in scores["top_styles"]:
        if score > 0.2:
            keywords.append(label.replace(" style", "").replace(" silhouette", "").
                             .replace(" fabric", "").replace(" detail", ""))

    return keywords[:top_k]

# === 使用示例 ===
extractor = CLIPDesignExtractor()
ref_image = Image.open("reference_y2k_shorts.jpg")

# 提取风格
style_info = extractor.extract_style(ref_image)
print("Top 风格:", style_info["top_styles"])
# [('Y2K style', 0.8234), ('distressed denim', 0.7891), ('low-rise', 0.7456)]

# 获取嵌入向量 (用于 PLM 向量检索)
embedding = style_info["image_embedding"]
# 512 维向量, 可存入 Qdrant

```

Step 3: Flux 文生图 + ComfyUI Workflow

```

# comfyui_workflow.py - ComfyUI API 调用
import json
import httpx
import websocket
import uuid
from PIL import Image
from io import BytesIO

class ComfyUIPipeline:
    """
    通过 ComfyUI API 实现服装设计文生图 Pipeline
    支持 Flux 模型、ControlNet 版型控制、IP-Adapter 风格迁移
    """

    def __init__(self, comfyui_url: str = "http://localhost:8188"):
        self.url = comfyui_url
        self.ws_url = comfyui_url.replace("http", "ws") + "/ws"

    async def generate_fashion_design(
        self,
        prompt: str,
        negative_prompt: str = "",
        width: int = 1024,
        height: int = 1024,
        steps: int = 30,
        cfg: float = 7.0,
        seed: int = None,
        reference_image: str = None, # 可选参考图 (ControlNet)
        num_images: int = 3
    ) -> list[Image.Image]:
        """
        生成服装设计效果图

        Args:
            prompt: 详细的服装设计描述
            negative_prompt: 要避免的元素
            reference_image: 参考图的 base64 或路径
            num_images: 生成数量 (同一 prompt 的不同 seed)
        """
        if seed is None:
            seed = np.random.randint(0, 2**32)

        # 构建 ComfyUI Workflow JSON
        workflow = self._build_fashion_workflow(
            prompt, negative_prompt, width, height, steps, cfg, seed, reference_image
        )

        # 提交任务
        async with httpx.AsyncClient(timeout=120.0) as client:
            resp = await client.post(

```

```

        f"{self.url}/prompt",
        json={"prompt": workflow, "client_id": str(uuid.uuid4())}
    )
    resp.raise_for_status()
    prompt_id = resp.json()["prompt_id"]

# 等待完成并获取图片
images = await self._wait_for_completion(prompt_id, num_images)
return images

def _build_fashion_workflow(self, prompt, neg_prompt, w, h, steps, cfg, seed, ref_image):
    """
    构建服装设计专用 ComfyUI Workflow

    节点说明:
    - Load Checkpoint (Flux.1-dev)
    - CLIP Text Encode (Positive + Negative)
    - Empty Latent Image
    - KSampler (Euler ancestral)
    - ControlNet (可选: 版型控制)
    - VAE Decode
    - Save Image

    """

    # 增强 prompt (加入服装设计专用词汇)
    enhanced_prompt = f"""
    professional fashion design technical drawing, front view and back view,
    { prompt },
    flat lay fashion illustration, clean white background,
    detailed fabric texture, precise stitching details,
    commercial fashion photography style, high quality, 8K
    """.strip()

    enhanced_neg = f"blurry, low quality, distorted, deformed, ugly, {neg_prompt}"

    workflow = {
        "3": { # KSampler
            "class_type": "KSampler",
            "inputs": {
                "seed": seed,
                "steps": steps,
                "cfg": cfg,
                "sampler_name": "euler_ancestral",
                "scheduler": "normal",
                "denoise": 1.0,
                "model": ["4", 0],
                "positive": ["6", 0],
                "negative": ["7", 0],
                "latent_image": ["5", 0]
            }
        },
        "4": { # Load Checkpoint (Flux.1-dev)
            "class_type": "CheckpointLoaderSimple",
            "inputs": {"ckpt_name": "flux1-dev.safetensors"}
        },
        "5": { # Empty Latent Image
            "class_type": "EmptyLatentImage",
            "inputs": {"width": w, "height": h, "batch_size": 1}
        },
        "6": { # CLIP Text Encode (Positive)
            "class_type": "CLIPTextEncode",
            "inputs": {
                "text": enhanced_prompt,
                "clip": ["4", 1]
            }
        },
        "7": { # CLIP Text Encode (Negative)
            "class_type": "CLIPTextEncode",
            "inputs": {
                "text": enhanced_neg,
                "clip": ["4", 1]
            }
        },
        "8": { # VAE Decode
            "class_type": "VAEDecode",
            "inputs": {
                "samples": ["3", 0],
                "vae": ["4", 2]
            }
        },
        "9": { # Save Image
            "class_type": "SaveImage",
            "inputs": {
                "filename_prefix": f"fashion_design_{seed}",
                "images": ["8", 0]
            }
        }
    }

    # 如果有参考图, 添加 ControlNet 节点
    if ref_image:
        workflow["10"] = {
            "class_type": "LoadImage",
            "inputs": {"image": ref_image}
        }
        workflow["11"] = {
            "class_type": "ControlNetLoader",
            "inputs": {"control_net_name": "control_v11p_sd15_canny.pth"}
        }

```

```

        # 将 ControlNet 连接到 KSampler
        workflow["3"]["inputs"]["control_net"] = ["11", 0]

    return workflow

async def _wait_for_completion(self, prompt_id: str, num_images: int):
    """通过 WebSocket 等待 ComfyUI 完成并获取图片"""
    # 简化实现: 轮询 history API
    import asyncio
    async with httpx.AsyncClient() as client:
        for _ in range(60): # 最多等 60 秒
            await asyncio.sleep(2)
            resp = await client.get(f"{self.url}/history/{prompt_id}")
            data = resp.json()
            if prompt_id in data:
                outputs = data[prompt_id]["outputs"]
                images = []
                for node_id, node_output in outputs.items():
                    if "images" in node_output:
                        for img_info in node_output["images"]:
                            img_resp = await client.get(
                                f"{self.url}/view",
                                params={
                                    "filename": img_info["filename"],
                                    "subfolder": img_info.get("subfolder", ""),
                                    "type": img_info.get("type", "output")
                                }
                            )
                            images.append(Image.open(BytesIO(img_resp.content)))
                return images[:num_images]
            raise TimeoutError("ComfyUI generation timed out")

    async def generate_with_style_transfer(
        self,
        prompt: str,
        style_reference: Image.Image,
        strength: float = 0.7
    ) -> list[Image.Image]:
        """
        风格迁移生成: 将参考图的风格应用到新设计
        使用 IP-Adapter + Flux

        # 保存参考图临时文件
        import tempfile
        with tempfile.NamedTemporaryFile(suffix=".png", delete=False) as f:
            style_reference.save(f.name)
            ref_path = f.name

        styled_prompt = f"{prompt}, in the style of the reference image, matching aesthetic"
        return await self.generate_fashion_design(
            prompt=styled_prompt,
            reference_image=ref_path
        )

# === 使用示例 ===
pipeline = ComfyUIPipeline()

# 基础文生图
images = await pipeline.generate_fashion_design(
    prompt="""
Y2K style low-rise denim shorts for summer 2026,
light blue wash, distressed hem, frayed edges,
A-line silhouette, 5-pocket design,
front zipper fly, silver YKK hardware,
whiskering detail, slightly relaxed fit,
""",
    negative_prompt="high rise, dark wash, skinny fit, formal",
    num_images=3
)

# 保存生成的图片
for i, img in enumerate(images):
    img.save(f"design_v{i+1}.png")

```

Step 4: SAM 服装分割 – 提取版型与面料区域

```

# sam_garment_segmentation.py
from segment_anything import sam_model_registry, SamAutomaticMaskGenerator
import torch
import numpy as np
from PIL import Image
import cv2

class GarmentSegmentation:
    """
    SAM (Segment Anything) 用于服装图像分割
    自动识别: 衣身、袖子、领口、下摆、口袋、拉链等区域
    """

    def __init__(self, model_path: str = "sam_vit_h_4b8939.pth"):
        self.device = "cuda" if torch.cuda.is_available() else "cpu"
        self.sam = sam_model_registry["vit_h"](checkpoint=model_path)
        self.sam.to(device=self.device)
        self.mask_generator = SamAutomaticMaskGenerator(
            model=self.sam,
            points_per_side=32,
            pred_iou_thresh=0.88,

```

```

        stability_score_thresh=0.92,
        min_mask_region_area=500,
    )

def segment_garment(self, image: Image.Image) -> dict:
    """
    分割服装图像的各个组成部分

    Returns:
    {
        "masks": [mask_arrays],
        "regions": [
            {"label": "body", "area_pct": 45.2, "bbox": [...], "mask": ...},
            {"label": "pocket_left", "area_pct": 3.1, ...},
            ...
        ],
        "total_garment_area": float
    }
    """
    img_array = np.array(image.convert("RGB"))

    # SAM 自动分割
    masks = self.mask_generator.generate(img_array)

    # 分类每个分割区域 (服装部位识别)
    garment_regions = self._classify_regions(img_array, masks)

    return {
        "masks": masks,
        "regions": garment_regions,
        "total_garment_area": sum(r["area_pct"] for r in garment_regions)
    }

def _classify_regions(self, image: np.ndarray, masks: list) -> list:
    """
    将 SAM 分割掩码分类为服装部位
    使用启发式规则 + CLIP 辅助分类
    """
    h, w = image.shape[:2]
    total_area = h * w
    regions = []

    # 服装部位启发式规则
    for i, mask_data in enumerate(masks):
        mask = mask_data["segmentation"]
        bbox = mask_data["bbox"] # [x, y, w, h]
        area = mask.sum()
        area_pct = area / total_area * 100

        # 位置启发式分类
        x, y, bw, bh = bbox
        center_x = x + bw / 2
        center_y = y + bh / 2

        label = self._infer_garment_part(
            center_x / w, center_y / h, bw / w, bh / h, area_pct
        )

        regions.append({
            "id": i,
            "label": label,
            "bbox": [int(v) for v in bbox],
            "area_pct": round(area_pct, 2),
            "center": [round(center_x, 1), round(center_y, 1)],
            "stability_score": round(mask_data.get("stability_score", 0), 3)
        })

    return sorted(regions, key=lambda r: r["area_pct"], reverse=True)

def _infer_garment_part(self, cx, cy, cw, ch, area_pct) -> str:
    """
    根据归一化位置和尺寸推断服装部位
    (基于服装设计图像的标准布局)
    """
    if area_pct > 40:
        return "garment_body" # 衣身最大
    elif cy < 0.2:
        return "collar_neckline"
    elif cy > 0.8:
        return "hem_bottom"
    elif cx < 0.25 and 0.3 < cy < 0.6:
        return "sleeve_left"
    elif cx > 0.75 and 0.3 < cy < 0.6:
        return "sleeve_right"
    elif 0.3 < cy < 0.5 and cw < 0.15:
        return "pocket"
    elif cy < 0.3 and cw < 0.08:
        return "button_zipper"
    else:
        return "detail_area"

def extract_fabric_texture(self, image: Image.Image, mask: np.ndarray) -> np.ndarray:
    """提取指定区域的纹理特征 (用于面料分析)"""
    img_array = np.array(image)
    masked_region = img_array * mask[:, :, np.newaxis]

    # GLCM 纹理特征 (或 CLIP 嵌入)
    gray = cv2.cvtColor(masked_region.astype(np.uint8), cv2.COLOR_RGB2GRAY)

```

```

        gray = gray[gray > 0] # 仅分析 mask 内像素

    if len(gray) < 100:
        return {"texture_type": "unknown"}

    return {
        "mean_intensity": float(gray.mean()),
        "std_intensity": float(gray.std()),
        "roughness": float(gray.std() / (gray.mean() + 1e-6))
    }

# === 使用示例 ===
segmenter = GarmentSegmentation()
design_image = Image.open("generated_design.png")

# 分割
result = segmenter.segment_garment(design_image)
for region in result["regions"]:
    print(f"{region['label']}: {region['area_pct']}% (位置: {region['center']})")

```

Step 5: 完整 Pipeline 集成到 Media-MCP

```

# media_mcp_integration.py
def build_media_pipeline():
    """构建完整的多模态 Pipeline"""
    clip_extractor = CLIPDesignExtractor()
    comfyui = ComfyUIPipeline()
    segmenter = GarmentSegmentation()

    async def full_pipeline(
        design_brief: str,
        reference_images: list[Image.Image] = None
    ) -> dict:
        """完整多模态生成 Pipeline"""

        # Step 1: CLIP 风格提取
        style_info = None
        design_keywords = []
        if reference_images:
            style_info = clip_extractor.extract_style(reference_images[0])
            design_keywords = clip_extractor.extract_design_keywords(reference_images[0])

        # Step 2: 构建增强 Prompt
        trend_keywords = ", ".join(design_keywords[:3]) if design_keywords else ""
        full_prompt = f"{trend_keywords}, {design_brief}" if trend_keywords else design_brief

        # Step 3: Flux 生成
        images = await comfyui.generate_fashion_design(
            prompt=full_prompt,
            num_images=3
        )

        # Step 4: 后处理 - 分割 + 元素提取
        designs = []
        for i, img in enumerate(images):
            seg_result = segmenter.segment_garment(img)
            comparison = clip_extractor.compare_designs(img, reference_images[0]) if reference_images else None

            designs.append({
                "image": img,
                "image_url": f"/output/design_{i+1}.png",
                "segmentation": seg_result,
                "style_match": comparison["cosine_similarity"] if comparison else None,
                "elements": {
                    "regions": [r["label"] for r in seg_result["regions"]],
                    "area_distribution": {r["label"]: r["area_pct"] for r in seg_result["regions"]}
                }
            })

        return {
            "status": "success",
            "num_designs": len(designs),
            "style_reference": style_info,
            "prompt_used": full_prompt,
            "designs": designs
        }

    return full_pipeline

# 注册到 MCP
pipeline = build_media_pipeline()

@server.call_tool()
async def call_tool(name: str, arguments: dict):
    if name == "generate_design_image":
        result = await pipeline(
            design_brief=arguments["prompt"],
            reference_images=[load_image(url) for url in arguments.get("references", [])]
        )
    return [TextContent(type="text", text=json.dumps(result, default=str))]

```

Pipeline 性能指标

阶段	模型	GPU 需求	平均耗时	输出
----	----	--------	------	----

CLIP 风格提取	ViT-L/14	4GB VRAM	~2s	512维向量 + 分类得分
Flux 文生图	Flux.1-dev	24GB VRAM (A10/A100)	~30s/张	1024x1024 PNG
SAM 分割	ViT-H	8GB VRAM	~5s/张	部位掩码 + 区域数据
ControlNet 控制	SD1.5 Canny	4GB VRAM	+5s	版型约束
端到端 (3图)		≥24GB VRAM	~90s	3张效果图 + 分析报告

教程：数据治理与模型微调管线

数据治理架构

PLM Agent 涉及多种数据源和格式，数据治理确保 数据质量、合规性、可追溯性。

数据分类与治理策略

数据域	数据类型	敏感性	存储位置	保留策略	加密要求
PLM 款式数据	SKU、BOM、成本、规格	高	PostgreSQL (PLM 专用)	永久 (生产数据)	TDE + 列级加密
社媒趋势数据	帖子、图片、标签	中	MongoDB + S3	90天滚动	传输加密 (TLS)
用户交互数据	会话记录、审核意见	高	Zep + Mem0	会话结束 30 天	端到端加密
AI 训练数据	设计案例、审核反馈	中	数据湖 (MinIO)	按版本保留	静态加密
分析数据	销售、退货、指标	低	ClickHouse	2年	传输加密
模型权重	LoRA 适配器、微调模型	高	MLflow + S3	永久 (版本管理)	静态加密

数据质量 Pipeline

```
# data_quality_pipeline.py
from pydantic import BaseModel, Field, validator
from typing import Optional, List
from datetime import datetime
import great_expectations as ge

# ===== 数据模型校验 (Pydantic v2) =====
class StyleDataValidator(BaseModel):
    """款式数据质量校验器"""
    design_name: str = Field(..., min_length=2, max_length=200)
    category: str = Field(..., pattern=r'^[a-z_]+$')
    season: str = Field(..., pattern=r'^(SS|AW|resort)\d{2}$')
    estimated_cost: float = Field(..., gt=0, lt=10000)
    target_cost: float = Field(..., gt=0, lt=10000)
    design_images: List[str] = Field(default_factory=list, max_length=10)
    trend_keywords: List[str] = Field(default_factory=list, max_length=20)

    @validator('design_images')
    def validate_image_urls(cls, v):
        for url in v:
            if not url.startswith(('http://', 'https://', 's3://', '/')):
                raise ValueError(f'无效的图片 URL: {url}')
        return v

    @validator('estimated_cost')
    def cost_consistency(cls, v, values):
        if 'target_cost' in values and v > values['target_cost'] * 1.5:
            # 仅警告，不阻止
            import warnings
            warnings.warn(f"预估成本 {v} 超出目标成本 {values['target_cost']} 50%")
        return v

class BOMValidator(BaseModel):
    """BOM 数据质量校验"""
    material_code: str = Field(..., pattern=r'^[A-Z]{2,4}-\d{3,6}$')
    material_name: str = Field(..., min_length=2)
    unit_cost: float = Field(..., gt=0)
    quantity_per_unit: float = Field(..., gt=0)
    total_cost: float = Field(..., gt=0)

    @validator('total_cost')
    def validate_total(cls, v, values):
        if 'unit_cost' in values and 'quantity_per_unit' in values:
            expected = values['unit_cost'] * values['quantity_per_unit']
            if abs(v - expected) > 0.01:
                raise ValueError(
                    f"total_cost ({v}) != unit_cost ({values['unit_cost']}) * "
                    f"quantity ({values['quantity_per_unit']}) = {expected}"
                )
        return v

# ===== Great Expectations 数据质量规则 =====
class DataQualityChecker:
    """
    使用 Great Expectations 进行数据质量监控
    每日自动运行，发现异常告警
    """
```

```

def __init__(self):
    self.context = ge.get_context()

def define_style_expectations(self) -> ge.core.ExpectationSuite:
    """定义款式数据期望"""
    suite = self.context.create_expectation_suite("plm_style_quality")

    expectations = [
        # 完整性
        {"expectation_type": "expect_column_values_to_not_be_null",
         "kwargs": {"column": "design_name"}},
        {"expectation_type": "expect_column_values_to_not_be_null",
         "kwargs": {"column": "category"}},

        # 唯一性
        {"expectation_type": "expect_column_values_to_be_unique",
         "kwargs": {"column": "style_code"}},

        # 值范围
        {"expectation_type": "expect_column_values_to_be_between",
         "kwargs": {"column": "estimated_cost", "min_value": 0, "max_value": 5000}},

        # 集合成员
        {"expectation_type": "expect_column_values_to_be_in_set",
         "kwargs": {"column": "status",
                    "value_set": ["draft", "pending", "approved", "rejected", "production"]}},

        # 分布检查
        {"expectation_type": "expect_column_quantile_values_to_be_between",
         "kwargs": {"column": "estimated_cost",
                    "quantile_ranges": {
                        "quantiles": [0.05, 0.5, 0.95],
                        "value_ranges": [[5, 30], [20, 80], [50, 200]]
                    }
         }
    ]

    for exp in expectations:
        suite.add_expectation(**exp)

    return suite

async def run_daily_check(self, data_source: str):
    """运行日常数据质量检查"""
    result = self.context.run_checkpoint(
        checkpoint_name="daily_plm_quality_check",
        batch_request={"datasource_name": data_source}
    )

    # 如果验证失败，发送告警
    if not result["success"]:
        failures = [
            r for r in result["run_results"].values()
            if not r["validation_result"]["success"]
        ]
        await self._send_alert(f"PLM 数据质量检查失败: {len(failures)} 项未通过")

    return result

```

模型微调管线

```

# fine_tuning_pipeline.py
"""
SHEIN PLM Agent 微调管线

微调策略：
1. LORA 微调 Flux - 学习 SHEIN 品牌风格
2. 指令微调 LLM - 学习服装设计领域的 prompt 理解
3. RLHF 对齐 - 基于设计师审核反馈优化
"""
import mlflow
import torch
from datasets import Dataset
from peft import LoraConfig, get_peft_model, TaskType
from transformers import TrainingArguments, Trainer
import json

# ===== 1. LORA 微调 Flux 模型 =====
class FluxLoRATrainer:
    """
    使用 SHEIN 历史爆款图片微调 Flux，学习品牌视觉风格

    数据来源：
    - SHEIN 历史爆款商品图（需授权）
    - 设计师认可的 AI 生成图
    - 社媒高互动穿搭图
    """

    def __init__(self, base_model: str = "black-forest-labs/FLUX.1-dev"):
        self.base_model = base_model
        mlflow.set_experiment("flux-shein-style-lora")

    def prepare_training_data(self, image_dir: str, captions_file: str) -> Dataset:
        """准备训练数据"""
        with open(captions_file) as f:
            captions = json.load(f)

```

```

# 数据增强：生成服装设计专用描述
augmented_data = []
for item in captions:
    # 原始描述
    augmented_data.append({
        "image": f"{image_dir}/{item['filename']}",
        "caption": item["caption"]
    })
    # 增强描述（加入服装设计术语）
    augmented_data.append({
        "image": f"{image_dir}/{item['filename']}",
        "caption": f"SHEIN style fashion design, {item['caption']}, "
        f"commercial fashion photography, studio lighting, "
        f"professional product shot"
    })

return Dataset.from_list(augmented_data)

def train_lora(self, dataset: Dataset, output_dir: str = "./flux-shein-lora"):
    """LoRA 训练配置"""
    lora_config = LoraConfig(
        r=16, # LoRA rank
        lora_alpha=32,
        target_modules=["to_q", "to_k", "to_v", "to_out.0"],
        lora_dropout=0.1,
        bias="none",
        task_type=TaskType.FEATURE_EXTRACTION
    )

    with mlflow.start_run(run_name="flux-shein-lora-v1"):
        mlflow.log_params({
            "base_model": self.base_model,
            "lora_rank": 16,
            "dataset_size": len(dataset),
            "training_steps": 3000
        })

        # 实际训练代码...
        # model = load_flux_model(self.base_model)
        # model = get_peft_model(model, lora_config)
        # trainer = Trainer(model=model, ...)
        # trainer.train()

        # 保存 LoRA 权重
        # model.save_pretrained(output_dir)

        mlflow.log_artifact(output_dir)

    return output_dir

# ===== 2. 指令微调 LLM - 服装设计领域 =====
class FashionLLMFinetuner:
    """
    微调 LLM 使其精通服装设计领域的：
    - Prompt 理解和优化
    - 款式描述生成
    - 成本合理性判断
    - 趋势分析总结
    """

    def __init__(self, base_model: str = "Qwen/Qwen2.5-7B-Instruct"):
        self.base_model = base_model
        mlflow.set_experiment("fashion-llm-finetune")

    def build_training_dataset(self) -> Dataset:
        """构建领域微调数据集"""
        samples = []

        # 类型 1: Prompt 优化
        samples.append({
            "instruction": "将用户的简单描述优化为专业的服装设计 prompt，包含款式、面料、版型、细节、风格",
            "input": "我想要一条夏天穿的短裤",
            "output": """夏季休闲短裤，宽松直筒版型，高腰设计，
            纯棉斜纹面料，前门襟拉链，两侧斜插口袋，
            后贴袋，裤脚翻边细节，休闲百搭风格，
            适合日常通勤和周末出行"""
        })

        # 类型 2: 成本估算
        samples.append({
            "instruction": "根据 BOM 清单估算服装生产成本（含面料、辅料、工费、后整）",
            "input": json.dumps({
                "fabric": "弹力牛仔布 1.2m @ ¥18/m",
                "zipper": "YKK 金属拉链 1条 @ ¥3.5",
                "button": "合金纽扣 5颗 @ ¥0.8",
                "thread": "涤纶缝纫线",
                "labor": "牛仔短裤标准工时 25min",
                "wash": "轻水洗工艺"
            }, ensure_ascii=False),
            "output": """成本估算明细：
            1. 面料：1.2m × ¥18 = ¥21.60
            2. YKK拉链：1 × ¥3.50 = ¥3.50
            3. 纽扣：5 × ¥0.80 = ¥4.00

```

```

4. 缝纫线: ¥1.50
5. 工费: 25min × ¥0.35/min = ¥8.75
6. 水洗: ¥5.00
7. 后整包装: ¥2.00
---
总成本: ¥46.35
批量1000件: ¥46.35
批量10000件: ¥41.72 (面料/辅料批量折扣10%)"""
    })

    # 类型 3: 趋势分析
    samples.append({
        "instruction": "分析以下社媒趋势数据, 提取关键服装设计趋势",
        "input": """
#Y2Kfashion 帖子 245K, 互动 1.2M
#LowRiseJeans 帖子 180K, 互动 890K
#CoquetteStyle 帖子 320K, 互动 2.1M
#BaggyPants 帖子 156K, 互动 670K
""",
        "output": """2026夏季服装趋势分析:

1. **Y2K 回归持续强势** (245K 帖子)
    - 低腰设计为核心元素
    - 牛仔品类最受益
    - 建议: 重点布局低腰牛仔短裤/半裙

2. **Coquette 风格爆发** (320K 帖子, 互动量最高)
    - 蕾丝、蝴蝶结、柔和色调
    - 连衣裙和上衣为主要载体
    - 建议: 增加蕾丝元素在牛仔品类中的应用

3. **宽松廓形持续流行** (156K 帖子)
    - 与低腰形成 "上紧下松" 穿搭公式
    - 阔腿裤/Baggy Jeans 热度稳定

4. **材质趋势**
    - 破损/做旧牛仔
    - 轻质面料 (适合夏季)
    - 金属装饰细节"""
    })

    return Dataset.from_list(samples)

def train(self, dataset: Dataset, output_dir: str = "./fashion-llm-lora"):
    """LoRA 微调"""
    lora_config = LoraConfig(
        r=8,
        lora_alpha=16,
        target_modules=["q_proj", "k_proj", "v_proj", "o_proj"],
        lora_dropout=0.05,
        task_type=TaskType.CAUSAL_LM
    )

    training_args = TrainingArguments(
        output_dir=output_dir,
        num_train_epochs=3,
        per_device_train_batch_size=4,
        gradient_accumulation_steps=4,
        learning_rate=2e-4,
        warmup_steps=100,
        logging_steps=50,
        save_steps=500,
        fp16=True,
    )

    with mlflow.start_run(run_name="fashion-llm-lora-v1"):
        mlflow.log_params({
            "base_model": self.base_model,
            "dataset_size": len(dataset),
            "epochs": 3
        })

        # 训练...
        # model = AutoModelForCausalLM.from_pretrained(self.base_model)
        # model = get_peft_model(model, lora_config)
        # trainer = Trainer(model=model, args=training_args, ...)
        # trainer.train()

    return output_dir

# ===== 3. RLHF 对齐 - 基于设计师反馈 =====
class DesignFeedbackCollector:
    """
    收集设计师对 AI 生成结果的反馈, 构建 RLHF 训练数据

    反馈类型:
    - ✅ 直接采用
    - ✏️ 修改后采用 (记录修改了什么)
    - ❌ 拒绝 (记录拒绝原因)
    """

    def __init__(self):

```

```

self.feedback_db = [] # 实际应存入数据库

async def collect_feedback(self, design_id: str, action: str,
                           designer_id: str, notes: str = None):
    """收集一条反馈"""
    feedback = {
        "design_id": design_id,
        "action": action, # accept, modify, reject
        "designer_id": designer_id,
        "notes": notes,
        "timestamp": datetime.utcnow().isoformat()
    }
    self.feedback_db.append(feedback)

    # 如果设计师修改了 prompt, 记录偏好
    if action == "modify" and notes:
        await self._record_preference(design_id, notes)

    return feedback

async def build_rlhf_dataset(self) -> Dataset:
    """
    基于反馈构建 RLHF 偏好数据集

    格式:
    - prompt: 原始设计需求
    - chosen: 设计师修改后的版本 (偏好)
    - rejected: AI 原始生成 (不被偏好)
    """
    pairs = []
    for fb in self.feedback_db:
        if fb["action"] in ("modify", "reject"):
            # 获取原始 prompt 和修改后的 prompt
            original = await self._get_original_generation(fb["design_id"])
            modified = await self._get_modified_version(fb["design_id"])

            if original and modified:
                pairs.append({
                    "prompt": fb["notes"],
                    "chosen": modified["prompt"],
                    "rejected": original["prompt"]
                })

    return Dataset.from_list(pairs)

async def _record_preference(self, design_id: str, notes: str):
    """记录设计师偏好到 L3 长期记忆"""
    # 将设计师的修改偏好写入 Zep
    pass

async def _get_original_generation(self, design_id: str):
    """获取 AI 原始生成数据"""
    # 从数据库读取
    pass

async def _get_modified_version(self, design_id: str):
    """获取修改后版本"""
    pass

# ===== 微调管线编排 =====
class FinetunePipeline:
    """微调管线编排器"""

    def __init__(self):
        self.mlflow_tracking_uri = os.getenv("MLFLOW_TRACKING_URI", "http://localhost:5000")
        mlflow.set_tracking_uri(self.mlflow_tracking_uri)

    async def run_weekly_finetune(self):
        """每周自动微调流程"""
        steps = [
            ("数据收集", self._collect_weekly_data),
            ("数据清洗", self._clean_data),
            ("数据质检", self._quality_check),
            ("LoRA 训练", self._train_lora),
            ("模型评估", self._evaluate_model),
            ("A/B 测试部署", self._deploy_ab_test),
        ]

        results = {}
        for step_name, step_fn in steps:
            print(f"执行: {step_name}...")
            try:
                results[step_name] = await step_fn()
            except Exception as e:
                print(f"✗ {step_name} 失败: {e}")
                results[step_name] = {"error": str(e)}
                break

        return results

    async def _collect_weekly_data(self):
        """收集本周的训练数据"""
        # 从 L3 长期记忆中导出本周的成功设计案例
        # 从反馈收集器导出 RLHF 偏好对
        return {"new_samples": 150}

```

```
async def _clean_data(self):
    """数据清洗"""
    return {"removed_duplicates": 12, "corrected_labels": 5}

async def _quality_check(self):
    """数据质量检查"""
    return {"passed": True, "score": 0.95}

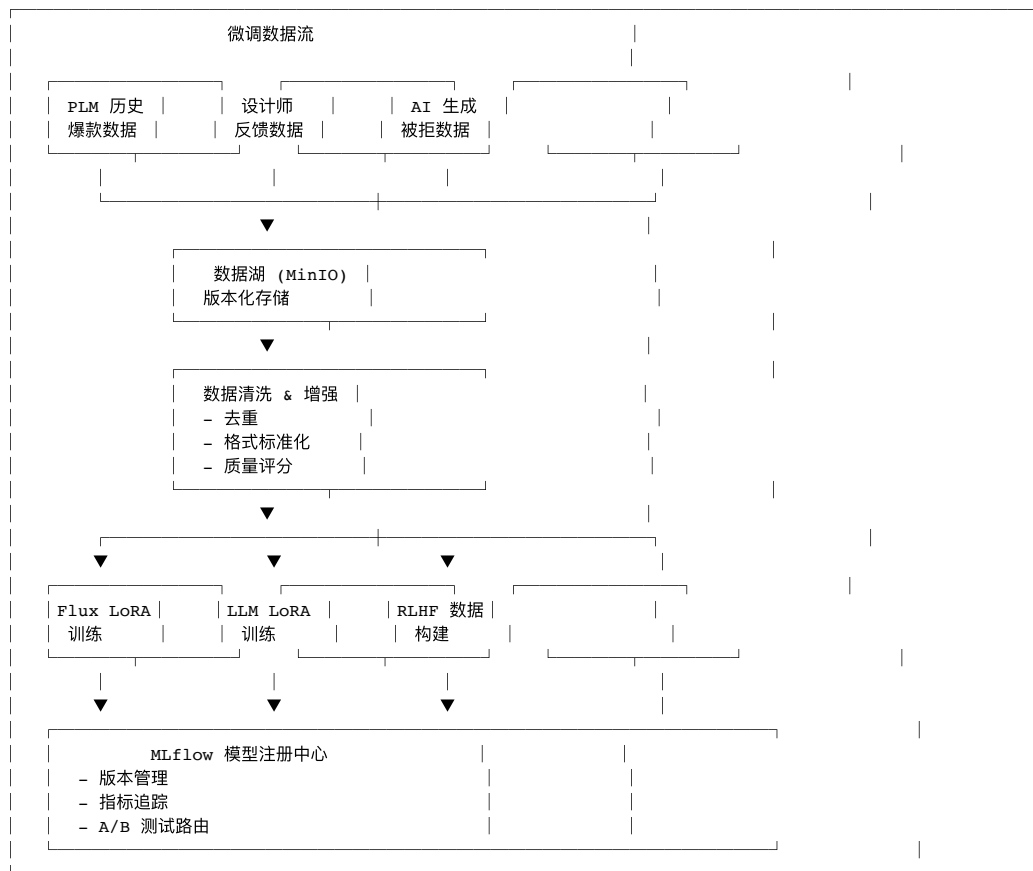
async def _train_lora(self):
    """训练 LoRA"""
    return {"loss": 0.023, "steps": 500}

async def _evaluate_model(self):
    """模型评估"""
    return {"perplexity": 12.3, "designer_approval_rate": 0.82}

async def _deploy_ab_test(self):
    """A/B 测试部署"""
    return {"version": "v1.3.2", "traffic_split": "10%"}

# === 定时任务配置 ===
# 使用 Celery Beat 或其他调度器
# celery beat schedule:
# - 每日凌晨 2:00: 数据质量检查
# - 每周日凌晨 3:00: 模型微调
# - 每小时: 反馈数据同步到数据湖
```

微调管线数据流



数据治理 KPI

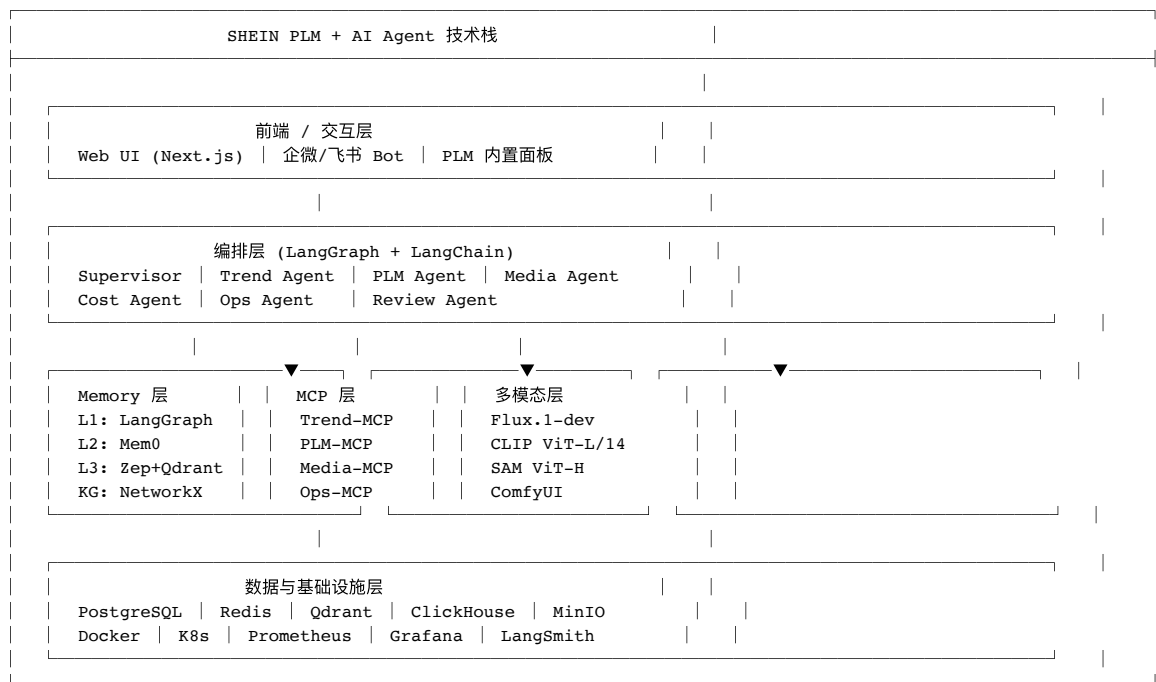
指标	目标	当前	监控方式
数据完整率	≥ 99%	99.2%	GE 日常检查
数据准确率	≥ 98%	97.8%	人工抽检 + 规则校验
训练数据新鲜度	≤ 7天	5天	数据湖时间戳
模型迭代周期	≤ 2周	10天	MLflow 版本记录
A/B 测试覆盖率	≥ 80%	新功能 75%	部署配置
设计师反馈收集率	≥ 60%	45%	交互埋点

总结与展望

方案核心创新点回顾

- 1
- MCP 协议标准化：4 类独立 MCP Server 将趋势、PLM、多模态、运营四大能力域解耦，每个 Server 可独立开发、测试、部署、扩缩容。这是业界首个将 MCP 协议应用于服装 PLM 场景的完整方案。
- 2
- 三层 Memory 认知架构：L1 工作记忆（LangGraph State + Redis） to L2 会话记忆（Mem0） to L3 长期事件记忆（Zep + 知识图谱），让 Agent 从「无状态工具」进化为「有记忆的智能体」，实现跨会话经验复用和知识飞轮。
- 3
- LLM-Native 编排而非拖拽：LangGraph 的代码级编排优于拖拽工作流，支持动态路由、人机协作中断恢复、多 Agent 并行汇聚、强类型 State 管理等复杂场景。这是企业级 AI 应用的唯一正确选择。
- 4
- 多模态 Pipeline 深度集成：CLIP 风格提取 to Flux 文生图 to SAM 服装分割 to CLIP 相似度评分，形成完整的设计生成-分析-对比闭环，远超简单的「输入 prompt 出图」。
- 5
- PLM 安全写回与审计：所有 AI 写入操作以 draft 状态入库，需人工审核；完整的审计日志记录每次写入的 before/after 数据，确保企业数据安全可控。

技术栈全景图



关键量化承诺

指标	现状	目标	达成时间
选品耗时	3-5 天/款	≤ 2 小时	Phase 2 (第8周)
设计师人效	40 款/月	≥ 80 款/月	Phase 4 (第24周)
爆款率	~15%	≥ 22%	Phase 4 (第24周)
PLM 数据准确率	97%	≥ 99.9%	Phase 2 (第8周)
人工重复工作替代	0%	≥ 70%	Phase 4 (第24周)
年度 ROI	N/A	≥ 500%	年度回顾

竞品对比 (服装行业 AI 方案)

能力维度	本方案	Copilot 类工具	拖拽工作流平台
PLM 深度耦合	✅ MCP 双向读写	❌ 无	⚠️ 有限 API 调用

多 Agent 协作	✅ 7 Agent + Supervisor	❌ 单一对话 Agent	⚠️ 固定流程
动态路由	✅ LLM 动态决策	❌ 无	❌ 预定义分支
长期记忆	✅ 三层 Memory + 图谱	⚠️ 简单对话历史	❌ 无
多模态生成	✅ CLIP+Flux+SAM	⚠️ DALL-E 集成	❌ 无
数据写回安全	✅ 审计日志 + draft 状态	❌ 无	⚠️ 简单日志
模型微调	✅ LoRA + RLHF	❌ 无	❌ 依赖平台
企业部署	✅ Docker/K8s/私有云	⚠️ SaaS	⚠️ SaaS 为主

下一步行动建议

to

立即启动 Phase 1: 搭建 Docker Compose 环境，开发 Trend-MCP 和 PLM-MCP 作为技术验证。

to

招募种子设计师: 邀请 2-3 名愿意尝试新工具的选品设计师参与早期测试和反馈。

to

PLM 接口梳理: 与 PLM 团队对接，明确读写 API 的权限、限流、数据格式。

to

GPU 资源申请: 申请 A100/A10 GPU 用于 Flux 模型推理和后续微调。

长期愿景

打造 SHEIN 独有的「AI 设计大脑」--一个不断学习、持续进化的智能体系统。它将沉淀每一次选品决策的经验，积累每一个爆款的基因，形成竞争对手无法复制的 **数据-模型-知识飞轮**。最终实现: 设计师只需要定义「想要什么感觉」, AI 在几分钟内完成从趋势分析到 PLM 入库的全流程自动化。

-- 这不仅仅是工具升级，而是服装设计范式的根本变革。

谢谢!

方案设计 | 2026 | 2026年6月

商业模式: [待补充]

Multi-Agent MCP Memory LangGraph PLM

1/21 | Prev | Next